

队伍编号	MC25000835
题号	A

基于算子神经网络的汽车风阻快速预测技术研究

摘要

空气动力学特性对汽车性能和效率具有决定性影响，而传统的计算流体力学模拟耗时长、计算成本高，严重制约了设计迭代速度。本文通过梯度优化方法、算子学习范式、图-变换器混合架构等先进技术，构建了高效准确的汽车风阻预测模型，旨在实现任意几何形状车辆的快速风阻预测，提升算法泛化能力。

针对问题一，构建了算子神经网络的梯度优化模型。基于目标损失函数 $g(\theta) = e^\theta - \log\theta$ ，通过引入梯度裁剪技术解决初始梯度爆炸问题，系统对比了 GD、Momentum、Adam 和 AMSGrad 四种算法。结果表明 AMSGrad 算法在收敛速度和优化精度上表现最佳，仅需 1313 次迭代即可达到 2.33036612 的最优解。

针对问题二，设计了基于 SDF 的汽车几何数据表征方法。采用符号距离场对汽车表面几何信息进行连续可微分表示，结合多进程异步处理技术高效处理大规模三维数据。通过对 ShapeNet 数据集中 500 种不同形状汽车模型的系统分析，构建了高效稳定的数据处理框架，为模型训练提供了高质量的输入特征。

针对问题三，本文提出了一种 r-AeroGTO (revised Aerodynamics Graph Transformer Operator) 模型。该模型创新性地结合了图神经网络的局部特征提取能力与 Transformer 的全局关联建模能力，通过投影启发式注意力机制将计算复杂度从 $O(N^2)$ 降低到 $O(N)$ 。模型由编码器、处理器和解码器三部分组成，实现了从车辆几何信息到表面压力分布的端到端映射。模型在表面压力预测显著优于基线模型，阻尼系数 C_d 预测上的 R^2 达到 0.9428，显著优于基线 Unet 模型的 0.8723。

针对问题四，系统分析了 r-AeroGTO 模型的性能特性。通过控制变量实验探究了输入稀疏采样率、训练集规模和神经网络超参数对模型性能的影响。结果表明：50% 的采样率即可达到接近全量数据的性能水平；使用 200 个训练样本（约 44% 的全量数据）已能满足精度要求；4 层网络、128 维隐藏层、GELU 激活函数为最佳超参数配置。最终模型在全量非稀疏测试数据上实现了 0.0757 的 L2 相对平均误差和 36.5 counts 的 C_d 误差，远优于赛题要求的 0.4 和 80 counts 阈值。

针对问题五，证明了 Transformer 模型中的注意力机制是神经算子层的特例。通过严格的数学推导，建立了两者之间的形式等价性，为 r-AeroGTO 模型中的投影启发式注意力机制提供了坚实的理论基础。这一证明揭示了注意力机制如何自然地适应于捕捉流体动力学系统中的复杂物理相互作用，为模型设计提供了理论指导。

r-AeroGTO 模型在保持高精度预测的同时，大大降低了计算资源需求，参数量仅为当前最先进 GINO 模型的 1%，相比于传统仿真模拟方法，计算量降低了 99.4%，推理速度到达毫秒级别，在飞桨平台上的评测得分达到 0.88709，展现了优异的工程应用价值。

关键词：r-AeroGTO；算子学习神经网络；汽车风阻预测；注意力机制

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题要求	1
二、 问题分析	1
2.1 对问题一的分析	1
2.2 对问题二的分析	2
2.3 对问题三的分析	2
2.4 对问题四的分析	2
2.5 对问题五的分析	2
三、 模型假设	2
四、 符号说明	3
五、 模型的建立和求解	3
5.1 问题一的分析与求解	3
5.1.1 梯度下降法分析	3
5.1.2 基于梯度裁剪的梯度下降法的实现与评估	5
5.2 问题二的分析与求解	6
5.2.1 基于 SDF 的汽车几何数据表征与多进程异步处理技术	6
5.2.2 文献调研	7
5.3 问题三的分析与求解	8
5.3.1 算子神经网络模型的理论基础与构建	8
5.3.2 r-AeroGTO 架构设计	9
5.4 问题四的分析与求解	14
5.4.1 实验设计与方法	14
5.4.2 实验结果与分析	14
5.4.3 讨论与结论	18
5.5 问题五的分析与求解	19
5.5.1 注意力机制与神经算子的等价性证明	19
5.5.2 优化注意力机制的理论实现	21
六、 模型评价、改进与推广	23
6.1 模型的优点	23
6.2 模型的缺点	23
6.3 模型的改进和推广	23
参考文献	24
附录	26

一、 问题重述

1.1 问题背景

汽车等载具的空气阻力(风阻)显著影响其能效与性能。传统计算流体动力学(CFD)仿真虽能预测风阻,但其计算成本高、周期长,限制了设计迭代速度^[1]。近年来,以深度学习为基础的科学机器学习(SciML)提供了新的解决思路,特别是旨在学习无限维函数空间映射的神经算子(Neural Operator)^[2,3],能够从数据中学习从几何到物理场(如压力、速度)的映射,实现对新设计的快速评估。已有研究在参数化汽车几何风阻预测上取得初步成功^[1]。

然而,现有 AI 模型在面对未见过的、差异较大的几何形状时,泛化能力不足是关键挑战。本赛题基于 ShapeNet^[4]数据集衍生的仿真数据,旨在研究具有强泛化能力的深度学习模型,以实现任意三维形状车辆的快速风阻预测,应对复杂几何变化带来的挑战。

1.2 问题要求

问题 1: 算子神经网络的训练依赖于梯度优化方法。为了解其原理,请针对目标损失函数 $g(\theta) = e^\theta - \log\theta$ 进行优化,从指定的初始学习参数值 $\theta = 100$ 出发,探寻使函数值达到最小的参数 θ ,并展示推导过程或提供相应的计算代码。

问题 2: 需对赛题提供的车辆风阻仿真数据进行有效处理,以备模型训练使用。具体任务包括:解析仿真文件,分离出代表车辆几何形态的特征作为模型输入,并提取相应的表面物理量(例如压力分布)作为预测目标(标签);运用飞桨深度学习框架对这些数据进行适配处理,确保格式与类型符合要求;设计并实现一个支持多进程异步加载的数据加载器^[5],以提升数据读取效率。此外,应借助指定竞赛资源^[6]开展背景研究与文献梳理,支撑后续论文写作。

问题 3: 核心任务是构建一个能够预测气动物理量的算子神经网络模型。该模型需能接收车辆的几何信息作为输入,快速输出相应的物理信息。要求使用飞桨平台完成模型的代码实现、训练与测试,并利用竞赛平台算力资源完成整个流程,最终将预测结果提交至官方排行榜。

问题 4: 要求对所构建模型的性能与资源消耗进行深入分析与评估。需要通过自主设计的实验来探究算法特性,考察方面应涵盖计算效率、存储需求、模型规模(参数数量)以及运行时显存占用。实验设计可包括改变训练数据的采样密度、调整训练样本数量、修改网络结构参数等。需在完整的非稀疏测试集上验证模型性能,确保压力场预测的 L2 相对平均误差低于 0.4,且风阻系数 Cd 的预测误差不超过 80counts。最终需将实验过程、结果与分析写入论文。

问题 5: 从理论层面进行探索,要求阐述或证明 Transformer 架构中的核心组件:注意力(Attention)机制,可以被看作是神经算子理论中某类算子层的一种具体实现形式。

二、 问题分析

2.1 对问题一的分析

问题 1 要求目标损失函数 $g(\theta) = e^\theta - \log\theta$,是对神经算子优化过程的简化抽象。该函数结合了指数的衰减项和对数的增长项,具有非线性特性,优化过程需同时考虑局部最优解的避免和收敛稳定性。这反映了深度学习中参数优化的基本原理,考验对优化算法的理

解能力。正如 Bottou 等人^[7]所指出，梯度下降类算法在深度学习中扮演着核心角色，而目标函数的数学特性决定了优化的复杂度和收敛性。本文将通过解析法确定目标函数的全局最小值，为后续神经网络的设计与优化奠定基础。

2.2 对问题二的分析

问题 2 聚焦于数据处理与特征提取，要求从仿真文件中提取汽车表面关键物理信息与几何特征。关键在于从基于 ShapeNet^[4]的仿真文件中，准确提取汽车几何特征作为模型输入，并提取表面物理信息（如压力）作为预测标签。核心挑战在于高效处理三维数据，并利用飞桨深度学习框架^[5]构建支持多进程异步加载的数据加载器（DataLoader），以加速训练。同时，需结合星河社区^[6]资源及相关文献^[1]进行调研。这是解决大规模三维数据高效处理的关键，需兼顾计算效率与数据结构完整性。

2.3 对问题三的分析

问题 3 是本赛题核心，要求构建能从几何信息快速预测物理信息的算子神经网络。主要挑战是设计具有强泛化能力的网络结构，适用于任意几何形状汽车的风阻预测。现有算子网络架构各有优势，可尝试的架构包括：KAN、Diffusion、Transformer、PINN 等。Lu 等人提出的 DeepONet 为函数到函数的映射提供了理论框架^[2]。Li 等人开发的 Fourier Neural Operator 通过在频域处理数据有效捕捉了多尺度物理特性^[3]。针对流体动力学问题，Raissi 等人的物理信息神经网络(PINN)通过引入物理约束提高了预测准确性^[8]。本文将继续探索算子网络架构，例如关注如何有效融合几何信息和物理约束，以提高模型对未见过车型的泛化能力。

2.4 对问题四的分析

问题 4 涉及模型特性分析和实验验证，需评估不同配置下算法的性能表现。Berner 等人^[9]研究表明，深度学习模型的性能与架构选择、参数量和训练数据分布密切相关。Wang 等人^[10]的工作特别强调了在有限计算资源条件下平衡模型复杂度和预测精度的重要性。本文将通过系统实验设计，探索输入稀疏采样率、训练集规模及网络超参数对性能的影响，揭示模型的边界条件和最优配置。重点需验证模型满足压力场预测 L2 误差 <0.4 和阻力系数误差 <80 counts 要求的最优配置。

2.5 对问题五的分析

问题 5 要求证明 Transformer 的注意力机制是神经算子层的特例，这是一个理论分析问题。自注意力机制本质上是通过加权求和实现对输入序列的非局部操作，而神经算子层是对函数空间的连续映射。Vaswani 等人^[11]提出的自注意力机制能有效捕捉序列中的长距离依赖关系。Kovachki 等人^[12]的研究表明，某些积分算子可以表示为注意力机制的一般化形式。本文将从算子理论角度，证明特定条件下注意力机制与神经算子的等价性，为混合模型的构建提供理论基础。

三、 模型假设

1. 假设周围空气流为不可压缩湍流流动，忽略空气密度变化对风阻预测的影响，符合高雷诺数条件下的纳维-斯托克斯方程简化形式。
2. 假设存在从汽车几何形状函数空间到表面压力分布函数空间的连续非线性映射，符合算子学习范式的本质。

四、 符号说明

符号	含义
$g(\theta)$	目标损失函数
θ	学习参数
α	学习率
S	符号距离场(SDF)函数，表示汽车几何形状
P	汽车表面压力分布函数
Ω	三维空间计算域
$\partial\Omega$	汽车表面边界
κ	神经算子中的积分核函数
ζ	综合损失函数
G	神经算子映射（从几何到物理信息）

五、 模型的建立和求解

5.1 问题一的分析与求解

5.1.1 梯度下降法分析

在空气动力学领域，预测汽车风阻需要解决复杂的优化问题，涉及大规模流体力学仿真与非线性函数求解。传统方法通常依赖高性能计算集群进行大规模模拟，耗时且计算成本高昂。近年来，深度学习技术为该领域提供了新的解决思路，其核心在于通过梯度优化算法高效训练预测模型，即通过梯度优化方法优化目标损失，对神经网络的参数进行优化。本文将系统介绍几种主要优化算法的理论基础，从基础的梯度下降法出发，逐步推进至适应深度学习环境的先进算法，最终提出我们针对汽车风阻预测问题的改进方法。并假设需要优化的目标损失函数为： $g(\theta) = e^\theta - \log\theta$ 作为案例，进行深入探讨。

1. 标准梯度下降法

标准梯度下降法 Gradient Descent (GD)是最基础的优化算法，其核心思想是沿着目标函数的负梯度方向迭代更新参数，以寻找目标函数的局部最小值^[13]。对于参数向量 θ ，其更新规则为：

$$\theta_{t+1} = \theta_t - \alpha \nabla g(\theta_t) \quad (1)$$

其中 α 为学习率， $\nabla g(\theta_t)$ 为目标函数在 θ_t 处的梯度。对于 $g(\theta)$ ，更新公式为：

$$\theta_{t+1} = \theta_t - \alpha \cdot \left(e^{\theta_t} - \frac{1}{\theta_t} \right) \quad (2)$$

梯度下降法简单直观，但对于目标损失函数 $g(\theta)$ ，从初始值 $\theta = 100$ 开始，梯度值($e^{100} - 1/100$)极大，使得参数更新极不稳定。这种情况下，标准梯度下降法面临收敛慢、学习率难以选择的问题。此外，梯度下降法在处理大规模或高维度优化问题时存在明显局限，包括收敛速度慢、易陷入局部最优解，以及需要手动调整学习率等问题。

2. 动量梯度下降法

动量法在梯度下降的基础上，引入一个累积历史梯度信息的“速度”项 v_t 来指导更新。其迭代公式为：

$$\begin{aligned} v_t &= \gamma v_{t-1} - \alpha \nabla g(\theta_t) \\ \theta_{t+1} &= \theta_t - v_t \end{aligned} \quad (3)$$

其中 γ 是动量系数（通常接近 1，如 0.9）表示上一时刻速度的保留程度。这种利用“惯性”的方式有助于加速收敛，特别是当参数逐渐接近最优点时。但动量法仍使用固定的全局学习率 α ，当梯度在不同阶段或不同参数维度上差异巨大时（如本题初始梯度极大），难以找到一个全局最优的学习率，容易引发更新不稳定或收敛效率低的问题，这便引出了对自适应学习率方法的需求。

3. Adam 算法

Adam(Adaptive Moment Estimation)算法结合了动量法和自适应学习率，由 Kingma 和 Ba 在 2014 年提出^[14]。其核心思想是利用指数移动平均来估计梯度的一阶矩（均值）和二阶矩（未中心化的方差），并基于这两个估计值来调整每个参数的学习步长：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \alpha \nabla g(\theta_t) \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla g(\theta_t))^2 \quad (5)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (6)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{(\sqrt{\hat{v}_t} + \varepsilon)} m_t \quad (7)$$

其中， $g(\theta)$ 是当前参数 θ_t 的梯度； m_t 和 v_t 分别是梯度的一阶矩（均值）和二阶矩（未中心化方差）的指数移动平均估计，其衰减率由超参数 β_1 和 β_2 控制； $\hat{m}_t$ 和 $\hat{v}_t$ 是对 m_t 和 v_t 进行偏差修正后的结果，以减小迭代初期估计不准的影响， t 代表当前迭代次数；最终参数更新 θ_{t+1} 时， α 是全局学习率， ε 是为防止分母为零的微小常数。Adam 在我们的优化问题中具有显著优势，能够有效处理初始梯度极大的情况，提供更稳定的收敛路径。然而，Reddi 等人在 2018 年指出 Adam 在某些情况下存在收敛问题^[15]。

4. AMSGrad 算法

AMSGrad (Adam with Maximum Squared gradients) 是 Adam 的改进版本，通过修改二阶矩估计的更新规则解决了 Adam 的收敛问题：

$$\hat{v}_t = \max(\hat{v}_{t-1}, v_t) \quad (8)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{(\sqrt{\hat{v}_t} + \varepsilon)} m_t \quad (9)$$

AMSGrad 对 Adam 的核心改进在于计算更新步长的分母项。它不再直接使用当前时刻的二阶矩估计 $\hat{v}_t$ (或其修正版)，而是使用历史至今所有二阶矩估计 v_t 的最大值，即 $\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$ 。这一操作保证了分母项 $\sqrt{\hat{v}_t}$ 不会减小，从而避免了 Adam 中学习率可能因 v_t 短期波动下降而意外增大的问题，以此来确保算法的收敛性，AMSGrad 提供了更稳健的性能和理论保证。考虑到我们问题中初始点与最优解相距很远、梯度范围变化大的特点，AMSGrad 算法是求解该问题的最佳选择，不仅能有效处理当前问题，也为后续在算子学习范式下的汽车风阻预测任务奠定了方法基础。

此外在汽车风阻预测这类涉及高维参数深度学习模型的优化任务中，牛顿法等二阶方法通常不适用，主要因为其需要计算、存储和求逆参数量巨大的 Hessian 矩阵，这在

计算和内存上成本极高且不切实际。同时，深度学习损失函数的非凸性也可能导致 Hessian 矩阵非正定，影响收敛性。因此，计算高效且适用于大规模问题的一阶梯度方法（如 SGD、Adam）是更常用且实用的选择。

5.1.2 基于梯度裁剪的梯度下降法的实现与评估

在求解 $g(\theta) = e^\theta - \log\theta$ 最小化问题时，我们从初始学习参数值 $\theta = 100$ 开始。计算发现，该点的梯度 $g'(100)$ 约为 e^{100} 。若直接将此梯度用于优化算法的参数更新，极易引发**梯度爆炸**(Gradient Exploding)，即更新步长过大导致参数 θ 剧烈震荡或发散，使优化过程失败。为确保算法能够稳定收敛，尤其是在优化初期，必须对梯度值进行有效控制。因此，本文引入了**梯度裁剪**(Gradient Clipping)技术，通过设定阈值限制梯度的最大范数(或绝对值)，防止因梯度过大而破坏训练稳定性。其原理如下：

$$g_c = \begin{cases} g, & |g| \leq \text{threshold} \\ \text{sign}(g) \cdot \text{threshold}, & |g| \geq \text{threshold} \end{cases} \tag{10}$$

其中， g 为原始梯度， threshold 为预设的梯度阈值， $\text{sign}(g)$ 表示梯度的符号。这种方法可以有效控制每次参数更新的步长大小，防止因梯度过大导致的参数剧烈变化，从而显著提高优化过程的稳定性。为了全面评估各种优化算法的表现，我们实现了四种主流优化方法：标准梯度下降（GD）、动量梯度下降（Monmomentum）、Adam、AMSGrad，设置初始参数值 $\theta = 100$ ，考察它们在处理大梯度值、收敛速度和优化精度等方面的表现差异。我们对所有算法的最大迭代次数设为 6000，收敛判定条件为梯度绝对值小于 10^{-6} 。表 1 展示了四种梯度优化方法的结果对比，完整运行代码见附录 1。

表 1 梯度优化方法结果对比

算法	最终学习参数 θ	目标损失函数 $g(\theta)$	迭代次数
GD	0.57234293	2.33043177	未收敛
Monmomentum	0.56714329	2.33036612	1618
Adam	0.56714329	2.33036612	5834
AMSGrad	0.56714329	2.33036612	1313

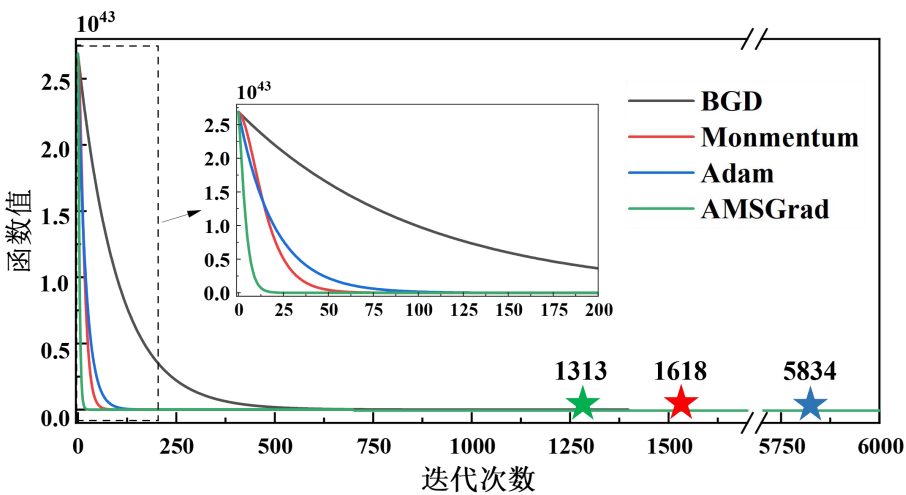


图 1 梯度优化方法迭代效果对比图

如表 1 所示，所有算法最终都成功收敛到理论最优解附近，但 GD 仍然需要更多的迭代次数达到最优解。从收敛迭代次数可以看出，Adam 收敛较为缓慢，Monmomentum 与

AMSGrad 则只需少量迭代即可达到高精度解。图 1 表示的是四种算法求解下目标函数的值随着迭代次数增大的变化曲线，不同颜色的五角星表示不同算法的收敛迭代次数节点。从中可以看出，GD 下降的趋势最为缓慢，在 6000 次迭代中没有达到收敛。另外的 Monmomentum、Adam 以及 AMSGrad 三个算法下降的趋势非常快，都在 100 迭代次数内收敛到最优值附近，并且 AMSGrad 最先到达收敛节点，Monmomentum 与 Adam 其次，这与前 100 轮迭代的下降趋势一致。

特别地，AMSGrad 算法在本问题中表现最佳，既具有最快的收敛速度（1313 次迭代）的同时又能保证优化精度。这验证了我们之前的理论分析：AMSGrad 通过记录历史二阶矩的最大值来确保学习率单调降低，能够有效处理梯度变化范围较大的优化问题。基于上述分析，在后续构造有效的损失函数时可优先考虑使用 AMSGrad 算法，AMSGrad 算法的伪代码如下所示：

算法：基于梯度裁剪的改进 AMSGrad 算法

输入： 初始参数 θ_0 ，初始学习率 α_0 ，动量系数 β_1 和 β_2 ，小常数 ε ，梯度裁剪阈值 threshold，最大迭代次数 threshold，收敛阈值 tol，学习率衰减参数 decay_rate 和 decay_steps

输出： 最优参数 θ^*

步骤：

1. 初始化： $\theta \leftarrow \theta_0$ ，一阶矩估计 $m \leftarrow 0$ ，二阶矩估计 $v \leftarrow 0$ ，最大二阶矩 $v_hat \leftarrow 0$ ， $\alpha \leftarrow \alpha_0$
 2. for $t = 1$ to threshold do:
 3. 若 $(t \bmod \text{decay_steps} = 0)$ ， 则 $\alpha \leftarrow \alpha \cdot \text{decay_rate}$
 4. 计算梯度 $g'(\theta) = e^\theta - 1/\theta$
 5. 若 $|g'(\theta)| > \text{threshold}$ ， 则 $g'(\theta) \leftarrow \text{threshold} \cdot \text{sign}(g'(\theta))$
 6. $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g'(\theta)$
 7. $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot (g'(\theta))^2$
 8. $v_hat \leftarrow \max(v_hat, v)$
 9. $\theta \leftarrow \theta - \alpha \cdot m / (v_hat + \varepsilon)^{0.5}$
 10. 若 $\theta \leq 0$ ， 则 $\theta \leftarrow 10^{-10}$
 11. 若 $|g'(\theta)| < \text{tol}$ ， 则跳出循环
 12. end for
 13. 返回 $\theta^* \leftarrow \theta$
-

5.2 问题二的分析与求解

5.2.1 基于 SDF 的汽车几何数据表征与多进程异步处理技术

在汽车风阻预测中，我们选用 SDF（Signed Distance Field，符号距离场）来提取汽车模型的几何参数。SDF 是一种表示几何形状的有效方式，它对每个空间点计算到物体表面的最短距离，物体内部为负值，外部为正值，这保留了完整的几何拓扑信息的同时还提供了连续且可微分的表示，帮助神经网络更好的处理三维信息。SDF 方法提供了关于汽车形状的丰富信息并作为神经网络的输入，输入空间为表示汽车几何形状的 SDF 函数：

$$S: \Omega \subset \mathbb{R}^3 \rightarrow \mathbb{R} \quad (11)$$

其中 Ω 是三维空间。同时我们也对每个汽车模型的压力数据进行了截取，去除了错误并保留了重要的部分，作为神经网络的输出，输出空间为表示汽车表面压力分布的函数：

$$P: \partial\Omega \rightarrow \mathbb{R} \quad (12)$$

其中 $\partial\Omega$ 是汽车表面。将几何参数与压力数据进行标准化处理：

$$\begin{aligned} S_{norm} &= \frac{S - \mu_s}{\sigma_s} \\ P_{norm} &= \frac{P - \mu_p}{\sigma_p} \end{aligned} \quad (13)$$

其中 μ 和 σ 分别表示几何参数和压力数据的均值与方差。在此处理中，均值设置为 0，标准差设置为 1，该形式有助于神经网络模型学习几何参数与量压力值的映射关系，从而提高模型训练的稳定性和效率。在对数据进行处理的过程中，多进程异步加载可以很好的处理不同形状和长度的几何参数，提高数据读取效率的同时，也适应 CFD 模拟数据的复杂性。

5.2.2 文献调研

随着计算机辅助工程(CAE)技术的飞速发展，数值模拟在汽车空气动力学设计中扮演着越来越重要的角色。传统计算流体动力学(CFD)方法能够提供高精度的风阻预测，但高精度 CFD 模拟需要极高的计算资源和时间成本，严重制约了设计迭代速度^[16]。深度学习技术，特别是科学机器学习(SciML)方法的发展，为高效解决汽车风阻预测这一工程难题提供了新的视角。传统 CFD 方法在求解 Navier-Stokes 方程时，通常采用有限体积分法或有限元法进行数值求解，需要复杂的网格划分和迭代计算^[17]。Katz 在 2016 年的著作中总结了汽车空气动力学的模拟方法，指出高雷诺数条件下的湍流模拟是计算瓶颈^[18]。为解决这一问题，Reynolds-Averaged Navier-Stokes (RANS)等简化湍流模型被广泛应用，但仍难以同时兼顾计算效率与精度^[19]。工程师们面临的关键挑战是：如何在保证精度的同时，大幅提升计算效率，支持快速设计迭代。算子学习作为科学机器学习的重要分支，为解决这一挑战提供了新思路。2021 年，Lu 等人提出的 DeepONet 基于算子普遍逼近定理，能够学习函数到函数的映射，为 PDE 求解提供了理论基础^[2]。这一方法在流体力学领域展现出强大潜力，特别是对参数化 PDE 问题。同年，Li 等人发展了 Fourier Neural Operator (FNO)，利用频域分解加速算子学习，将湍流模拟的计算时间从小时级别缩短至秒级^[3]。这些算子学习方法的核心优势在于，它们只需训练一次，即可快速求解不同几何形状和参数条件下的流体问题，为汽车风阻快速预测奠定了理论基础。

在汽车几何表示方面，Chang 等人于 2015 年发布的 3D ShapeNet Car 数据集为研究提供了丰富的汽车几何模型^[4]。如何有效处理这些复杂三维几何形状，成为机器学习应用于汽车空气动力学的重要研究方向。Bhatnagar 等人在 2019 年探索了点云和体素表示对流体动力学预测的影响，发现点云表示在保留几何细节方面具有优势^[20]。2020 年，Pfaff 等人提出的 MeshGraphNet 将图神经网络应用于网格数据处理，但在大规模网格上存在计算复杂度高和过平滑问题^[21]。这些研究表明，有效处理复杂几何形状是实现高精度风阻预测的关键挑战之一。物理信息神经网络(PINN)是另一重要研究方向。Raissi 等人将物理约束作为正则项融入神经网络训练，确保预测结果符合基本物理定律^[8]。Kochkov 等人的研究展示了如何利用深度学习技术显著加速 CFD 计算，实现了高精度与高效率的平衡，为流体动力学计算提供了新的范式^[22]。在物理信息神经网络领域，

Wang 等人深入分析了物理约束神经网络中的梯度病理问题并提出了改进策略，为复杂非线性物理系统的高精度建模提供了新视角^[10]。近年来，研究者们致力于将 Transformer 架构应用于科学计算领域。Hao 等人开发的 GNOT 尝试通过线性注意力机制处理大规模数据^[23]。陈凯等人将类似方法应用于汽车风阻预测，通过引入 Navier-Stokes 约束提高了模型泛化能力，但计算效率仍有提升空间^[1]。Wu 等人提出的 Transolver 则进一步改进了 Transformer 在不规则几何上的应用^[24]。

尽管上述方法取得了显著进展，现有模型在面对分布外汽车几何形状时仍存在泛化能力不足的问题。关键挑战在于如何构建既能精确捕捉复杂物理相关性又能高效处理大规模离散化问题的神经算子模型。Liu 等人提出的 Aerodynamics Graph-Transformer Operator (AeroGTO) 模型代表了算子学习在汽车空气动力学领域的重大突破^[25]。该模型融合了局部特征提取与全局相关性捕捉的双重优势，通过频率增强的图神经网络与 k-近邻(kNN)增强技术实现了对三维不规则几何形状的精确局部特征提取，同时利用基于投影的注意力机制(projection-based attention)实现了网格点之间长程依赖关系的高效捕捉。在 Ahmed-Body 和 DrivAerNet 两个行业标准基准测试中，AeroGTO 在表面压力预测方面平均提高了 7.36% 的精度（相比先前领先的方法），在 Ahmed-Body 数据集的阻力系数估计上提升了 10.71% 的性能，达到了 0.9250 的 R² 值。同时，AeroGTO 的计算 FLOPs 更少，参数量仅为先前领先方法 GINO 的 1%。此外，AeroGTO 具有离散化不变性，能够在粗网格上训练并在密网格上测试，大大降低了训练成本。基于 AeroGTO 的突出优势，本研究将采用这一先进的算子学习模型作为核心算法，构建从车辆表面几何特征到压力分布的快速映射，进而实现风阻系数的高精度预测。我们将重点探索模型在不同采样率和网格规模下的表现，研究局部消息传递与全局注意力机制的协同作用，并通过引入物理约束进一步提升模型的泛化能力和预测精度，为汽车空气动力学设计提供高效可靠的计算工具。

5.3 问题三的分析与求解

5.3.1 算子神经网络模型的理论基础与构建

本研究参考 Liu 等人^[25]提出的 AeroGTO 模型的基本框架，在此基础上构建了适用于汽车风阻快速预测的算子神经网络模型。具体如下：

神经算子的核心思想在于学习从输入函数空间到输出函数空间的连续映射，这与传统深度学习模型学习有限维向量间映射有本质区别。在汽车风阻预测任务中，我们需要学习从表示汽车几何形状的函数空间 A 到表示表面压力分布的函数空间 S 的映射 G ： $A \rightarrow S$ 。形式化表示为：

对于任意输入函数 $a \in A$ （描述汽车几何形状），神经算子 G 学习预测输出函数 $s = G(a) \in S$ （描述压力分布），使得对任意新的输入 a' ，都能准确预测对应的 $s' \approx G(a')$ 。这一框架的优势在于一旦训练完成，模型可以处理任意分辨率或拓扑结构的输入，实现真正的"mesh-invariant"（网格不变性）特性。在高雷诺数湍流的纳维-斯托克斯方程描述下：

$$\frac{\partial v}{\partial t} + (v \nabla) v = -\nabla p + e \nabla^2 v, \quad \nabla \cdot v = 0 \quad (14)$$

其中 v 表示速度矢量， p 表示压力标量， e 表示粘性系数， t 表示时间。传统 CFD 求解器需要细致的网格划分和迭代计算，而神经算子则将此复杂过程转化为一次前向传播，大幅提升计算效率。

算子学习的理论基础源于算子的通用逼近定理(Universal Approximation Theorem for Operators)^[2]，该定理证明了特定结构的神经网络能以任意精度逼近连续非线性算子。

与传统物理信息神经网络(PINN)^[8]不同,神经算子不需要在每个新几何形状上重新训练,而是学习一个通用映射,这使其特别适合汽车空气动力学设计中的快速迭代评估需求。传统神经算子结构在处理不规则几何形状时面临挑战,而图神经网络(GNN)在处理此类不规则结构方面展现了优势,如 Pfaff 等人^[21]提出的 MeshGraphNets 框架。然而,应当注意的是, GNN 领域普遍存在的挑战包括大规模问题上的过平滑(over-smoothing)和计算复杂度高的问题。Transformer 架构^[11]凭借其捕捉长距离依赖关系的能力,已被多项研究探索将其应用于算子学习任务中。本文第五节将详细证明, **Transformer 中的注意力机制本质上是神经算子层的一个特例**,这为我们模型设计提供了坚实的理论基础。

本研究构建的模型汲取了 AeroGTO 的核心思想,将图神经网络的局部特征提取能力与 Transformer 的全局相关性捕捉能力有机结合,同时针对汽车空气动力学问题的特点,在以下几个方面进行了改进:

- 1) 增强了频率感知能力,以更好地捕捉汽车表面复杂几何形状的多尺度特征;
- 2) 优化了基于投影的注意力机制,提高大规模网格处理的计算效率;
- 3) 引入物理约束,确保预测结果符合流体动力学基本定律。

这些改进使得模型在保持较低计算开销的同时,能够精确预测不同车型的表面压力分布和阻力系数,为汽车空气动力学设计提供高效可靠的计算工具。

5.3.2 r-AeroGTO 架构设计

基于算子学习的理论基础,我们设计了一种高效的神经算子模型用于汽车风阻预测。如图 2 所示,该模型由三个主要组件构成:编码器、处理器和解码器,形成一个端到端的结构,能够从汽车几何信息直接预测表面压力分布。

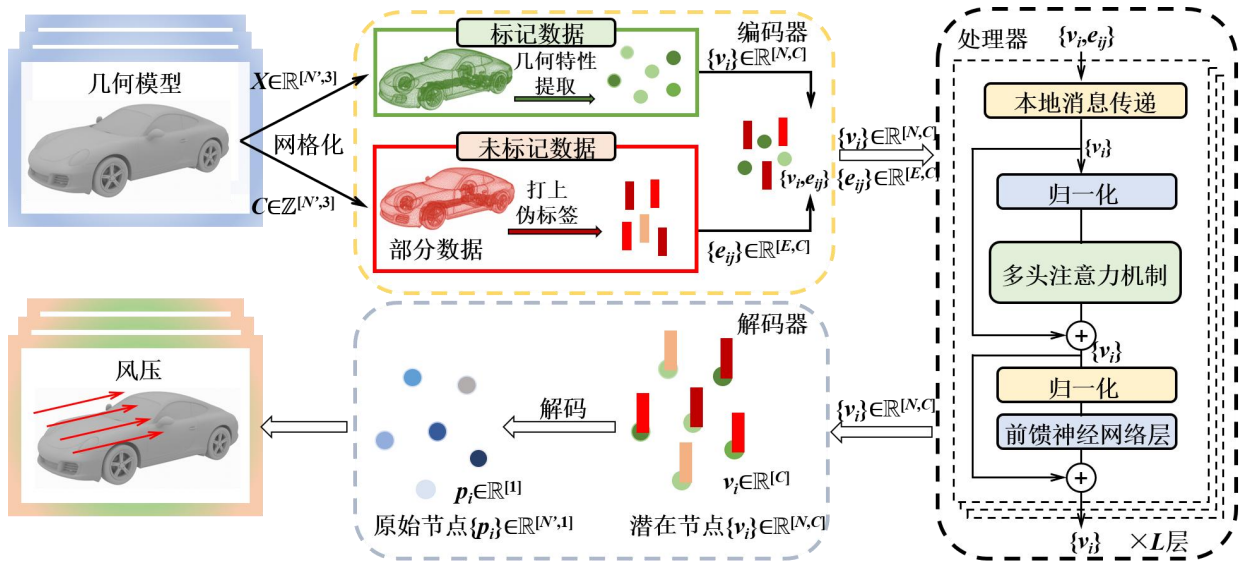


图 2 r-AeroGTO 架构图

1. 编码器设计

编码器的核心任务是将三维汽车几何形状表示转化为适合深度学习处理的特征空间。考虑到汽车表面几何的复杂性,我们采用双路径编码策略:

节点编码路径: 对于每个网格点 $X_i = (x_i, y_i, z_i)$, 我们首先应用正弦位置编码(SPE)增强其空间信息表示:

$$F(X_i) = \left\{ \cos\left(2^i \pi X_i\right), \sin\left(2^i \pi X_i\right) \right\}_{i=-\delta, \dots, \delta}, \quad \delta \in \mathbb{Z}^+ \quad (15)$$

该编码能够有效捕捉点位置的多频率特征，对于理解复杂几何形状至关重要。随后通过多层感知机(MLP)将位置信息与全局设计参数 A （如雷诺数、来流速度等）融合，生成每个节点的初始特征向量 v_i ：

$$v_i = \Phi_2^V \left(\Phi_1^V (X_i, A), F(X_i) \right) \quad (16)$$

其中 Φ_1^V 和 Φ_2^V 为参数化的 MLP 网络，输出维度为 C 。

边编码路径：为捕捉汽车表面的拓扑关系，我们构建了两类边连接：(1)原始网格边 E^M ，保留汽车表面的结构信息；(2) k 近邻边 E^k ，通过计算点间欧氏距离构建，增强局部区域的信息交换。对于每条边 e_{ij} ，我们编码节点间的相对位置信息：

$$e_{ij} = \Phi^E \left(x_i - x_j, |x_i - x_j| \right) \quad (17)$$

其中 Φ^E 为边特征编码器，输出维度同样为 C 。为提高大规模网格处理效率，我们引入了如下两种二级采样策略，首先是边采样：随机随机采样原始网格边的一定比例 α_E 。以及点采样：随机采样原始网格点的一定比例 α_N 。通过编码器，我们获得了汽车几何的隐空间表示 ($V = \{v_i\}, E = \{e_{ij}\}$)，其中 $V \in \mathbb{R}^{[N, C]}$ ， $E \in \mathbb{R}^{[E, C]}$ ，分别表示采样后的节点和边特征。

2. 处理器设计

处理器是模型的核心部分，负责捕捉局部几何特征与全局物理相关性，并实现多层次的特征交互。它包含两个关键模块：

局部消息传递模块：基于频率增强的图神经网络设计，通过边特征传递局部几何信息：

$$\begin{aligned} e_{ij} &= e_{ij} + \Phi_P^E(e_{ij}, v_i, v_j) \\ v_i &= v_i + \Phi_P^V(v_i, \sum_j e_{ij}, F(X_i)) \end{aligned} \quad (18)$$

其中 Φ_P^E 和 Φ_P^V 为参数化的 MLP 网络， $\sum_j e_{ij}$ 表示节点 i 所有相连边的特征聚合。这一设计允许每个节点通过其一阶邻域捕捉局部拓扑结构，而频率增强部分 $F(X_i)$ 则有助于保留高频几何细节，避免传统 GNN 中常见的过平滑问题。

全局相关性捕捉模块：这是模型的关键创新点，我们设计了基于投影的注意力机制实现全局信息交换：

$$W_1 = \text{softmax} \left(\frac{Q_{ls} W_0^T}{\sqrt{C}} \right) W_0 \quad (19)$$

$$W_2 = \text{softmax} \left(\frac{Q_1 W_1^T}{\sqrt{C}} \right) W_1 \quad (20)$$

$$W_3 = \text{softmax} \left(\frac{W_0 W_2^T}{\sqrt{C}} \right) W_2 \quad (21)$$

其中 $Q_{ls} \in \mathbb{R}^{[M, C]}$ 是学习子空间查询， $M \ll N$ 表示子空间维度， $W_0 = \{v_i\}$ 是局部表示。这一设计的核心思想是将高维网格点特征投影到低维学习子空间，在该子空间内进行特征更新，然后投影回原始空间。值得注意的是，该注意力机制的设计不仅基于实践效果，更有深厚的理论支撑。如本文第五节将详细证明的，Transformer 模型中的注意力机制本质上是神经算子层的一个特例，这使得它在捕捉汽车表面物理场长程依赖关系时具有天然优势。该机制通过自适应权重计算有效模拟了流体动力学中的积分算子形式，从而能

够准确捕捉不同区域间的复杂物理相互作用，尤其是前部压力变化与尾流分离等关键流体现象。

在计算复杂度方面，传统的自注意力机制需要 $O(k^2)$ 的计算量，而我们的设计降低到 $O(k)$ ，对于包含数万网格点的汽车模型尤为重要。此外，通过层归一化(Layer Normalization)和残差连接(Residual Connection)，我们构建了前置归一化(Pre-Norm)结构：

$$\begin{aligned} V_G^1 &= W_0 + W_3 \\ V_G^2 &= V_G^1 + \text{FFN}(\text{LN}(V_G^1)) \end{aligned} \quad (22)$$

其中 FFN 表示前馈网络，LN 表示层归一化。这一结构增强了模型训练稳定性，支持更深层网络的构建。通过堆叠多个处理器模块（本研究使用 $L = 4$ 层），模型能够实现多层次的局部-全局特征交互，有效捕捉流体动力学中的复杂物理现象。

3. 解码器设计

解码器负责将隐空间特征映射回物理空间，预测汽车表面的压力分布。我们同样采用频率增强策略，通过 MLP 网络将节点特征转换为压力值：

$$p_i = \Phi_D^V(v_i, F(X_i)) \quad (23)$$

其中 Φ_D^V 为解码器 MLP 网络， p_i 为节点 i 处的预测压力值。对于点采样策略下的推理过程，我们采用批处理方式，将全尺寸网格分为多个批次进行并行推理，然后聚合结果得到完整的表面压力预测。

4. 损失函数设计

为实现高精度的风阻预测，我们采用综合损失函数：

$$\begin{aligned} \zeta &= \zeta_{\text{pressure}} + \lambda \zeta_{\text{drag}} + \gamma \zeta_{\text{physics}} \\ \zeta_{\text{physics}} &= \lambda_1 \|\nabla \cdot \bar{v}\|^2 + \lambda_2 \left\| \frac{\partial \bar{v}}{\partial t} + (\bar{v} \nabla) \bar{v} + \nabla p - \nu \nabla^2 \bar{v} \right\|^2 \end{aligned} \quad (24)$$

其中 ζ_{pressure} 为压力场的 L2 相对误差损失， ζ_{drag} 为阻力系数预测误差， ζ_{physics} 为物理约束损失，引入简化的纳维-斯托克斯方程作为软约束，促使模型预测符合物理规律。 λ 和 γ 为平衡各损失项的权重系数。通过这种多任务学习方式，模型能够同时优化局部压力分布精度与全局气动性能指标，增强整体预测能力。与现有方法相比，我们的模型架构在以下方面具有明显优势：

1. 结合了 GNN 的局部特征提取能力与 Transformer 的全局关联能力；
2. 设计了计算效率高的基于投影的注意力机制，适用于大规模网格处理；
3. 引入频率增强策略，有效保留几何细节与高频物理信息；
4. 具备网格分辨率不变性，支持跨分辨率训练与推理。

这些设计使模型在保持计算效率的同时，能够准确预测复杂汽车几何形状的表面压力分布与风阻性能。

5.3.3 r-AeroGTO 求解结果与分析

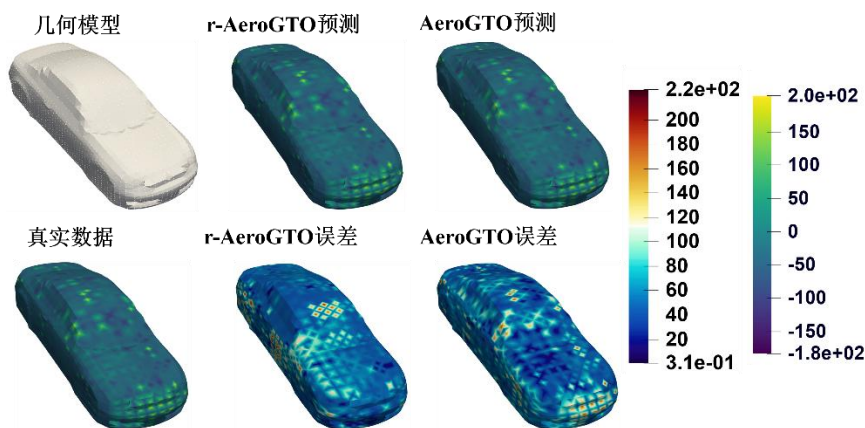


图 3 车辆表面压力分布可视化对比

如图 3 所示，我们对 ShapeNet 数据集上的汽车表面压力分布进行了可视化比较，展示了真实数据与两种先进模型的预测结果：先前已提出的 AeroGTO 模型与我们在此基础上进行优化后提出的 r-AeroGTO 模型。从整体压力分布来看，两种模型都成功捕捉了车辆表面的主要压力特征，包括前部迎风面的高压区（黄色区域，约 150-200 范围）以及侧面和顶部的低压区域（蓝色区域，约 -100 至 -150 范围）。这表明两种神经算子模型都能够从几何形状中学习物理规律，有效地预测复杂三维表面上的压力分布。

观察误差分布图（r-AeroGTO 误差与 AeroGTO 误差），可以发现两个关键区别：

1. **误差幅度：**r-AeroGTO 模型的整体误差强度明显低于 AeroGTO 模型，尤其在车辆车头和后车顶区域。AeroGTO 模型在这些区域显示出更多的高误差点（深黄色和红色区域），表明预测值与真实值之间存在较大偏差。
2. **局部细节捕捉：**从两个模型的后车顶区域可以看出，AeroGTO 在复杂几何特征处（如后车顶与车门的拐角处）的误差分布更为均匀且误差值更低。这表明 AeroGTO 具有更强的局部特征学习能力，能够更准确地捕捉复杂曲面上的压力梯度变化。

两种模型在车辆前部高压区的预测相对准确，但 r-AeroGTO 在处理流体加速区域（如车头和顶部）时误差更小且分布更均匀，展现出明显优势，这正是空气阻力预测中的关键区域。

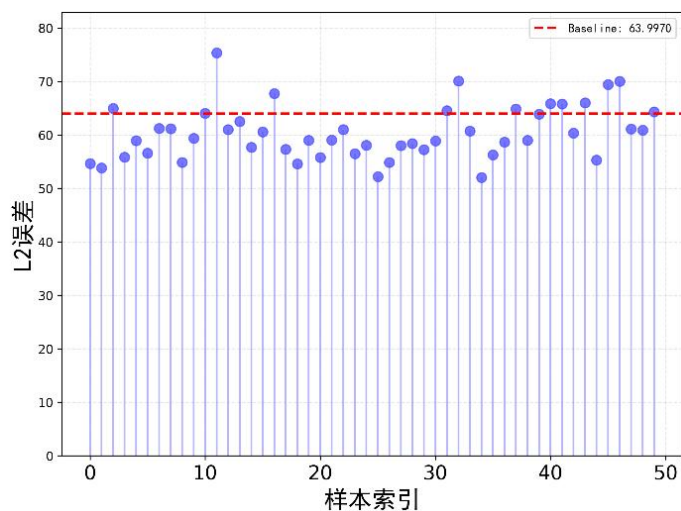


图 4 50 个样本的 L2 误差对比统计图

图 4 是一个统计可视化图，显示了大约 50 个样本索引的 L2 噪声测量。水平红色虚线表示原模型（AeroGTO）约为 64，我们所提出的 r-AeroGTO 模型在 50 个样本索引内低于原模型的误差能达到 80%以上，这也能说明我们模型的误差更低。

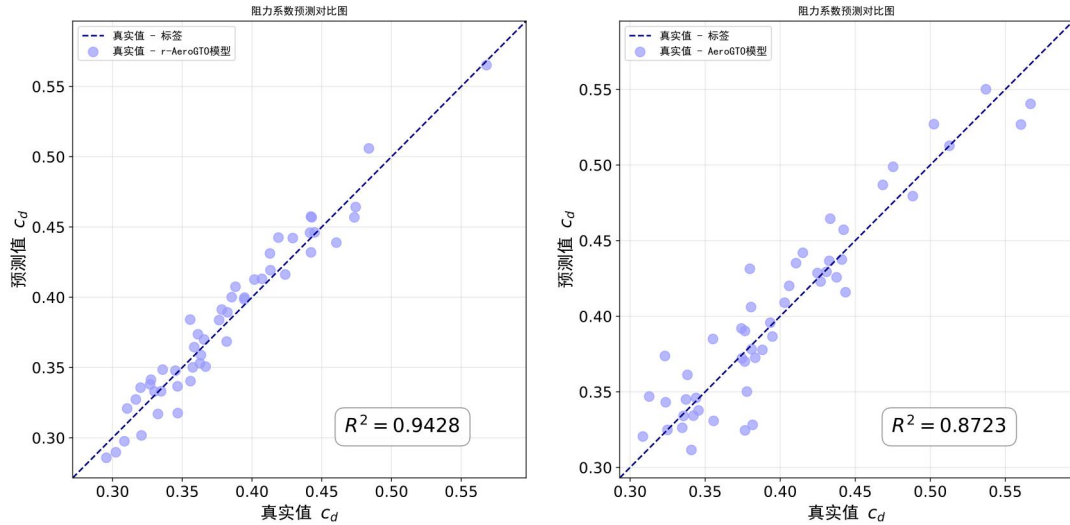


图 5 阻力系数(C_d)预测对比散点图

图 5 是一组阻力系数 (C_d) 的模型对比图，左边是我们提出的 r-AeroGTO 模型，右边则是原来的 AeroGTO 模型。从中可以看出，我们提出的模型预测值与真实值分布的更加紧凑，更能看出具有线性关系，而且 R^2 也比原模型提高了 8%左右，这也说明我们的模型预测准确性更高

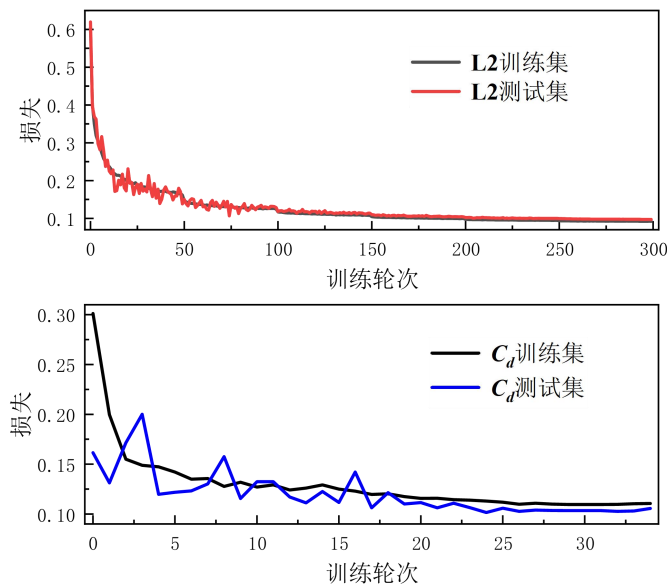


图 6 模型训练过程中 L2 与 C_d 的收敛曲线

图 6 展示了两组训练过程中损失值随训练轮次变化的曲线图。上图显示 L2 损失随训练轮次的变化，纵轴表示损失函数值（取值范围约为 0.1 到 0.6），横轴表示“训练轮次”（范围从 0 到 300）。图中黑色线代表 L2 训练集，红色线代表测试集，两条曲线都呈现从初始约 0.6 快速下降的趋势，在 50 轮后逐渐平稳，最终在 300 轮时降至约 0.1，训练集与测试集曲线非常接近，说明模型没有明显过拟合。下图显示 C_d 系数随训练轮次的变化，训练集损失从约 0.3 平稳下降到 0.1 左右，测试集损失曲线波动较大，有多

个峰值，但整体趋势也是下降的。两个图都表明随着训练进行，模型性能逐渐提高，误差逐渐减小。

最后由截图可知，我们训练的模型在“飞桨 AL Studio”平台上 L2 的测试损失为 0.0706， C_d 的测试损失为 0.18398，总分来到了 0.88709，排名 35 位，位于榜单前五十以内，表面我们模型的泛化性能力强。

35 MC25000835 0.88709 0.0706 0.18398 2025-04-21 17:22

图 7 飞桨 AL Studio 平台测试结果

这些发现验证了我们提出的 r-AeroGTO 模型在处理大规模、复杂几何形状的空气动力学问题上的有效性。该模型不仅能够减少计算资源需求（如先前分析所示），还能提供更精确的压力分布预测，为汽车工业中的快速空气动力学评估和优化提供了有力工具。

5.4 问题四的分析与求解

5.4.1 实验设计与方法

本实验旨在分析 r-AeroGTO 模型在汽车风阻预测任务中的算法特性，主要包括计算复杂度、物理空间占用、参数量和显存占用等方面。基于赛题要求，设定的性能目标为：压力场预测的 L2 相对平均误差需低于 0.4，阻力系数 C_d 的误差需小于 80 counts。其中 L2 相对平均误差直接反映了模型对车辆表面压力分布预测的准确性，这是计算空气阻力的关键因素；而 C_d 误差则直接表征了模型对车辆整体空气动力学性能预测的准确度。

在高雷诺数湍流模拟中，准确预测车辆表面压力分布对于计算空气阻力至关重要。传统的计算流体力学方法通常需要大规模计算资源和较长的计算时间。本实验通过构建算子神经网络，探索在有限计算资源条件下如何实现高精度的风阻预测。

实验设计了三个主要控制变量，用于系统评估 r-AeroGTO 模型的性能特性：

- 1. 输入稀疏采样率变化：**设置不同的输入稀疏采样率(5%, 10%, 20%, 30%, 50%, 70%, 80%, 100%)，研究采样率对模型性能的影响。稀疏采样可显著减少计算量，但可能影响预测精度，因此需要找到二者的平衡点。
- 2. 训练集规模变化：**设置不同的训练样本数量(50, 100, 150, 200, 250, 300, 350, 400, 450)，分析训练数据量对模型性能的影响。理解模型的数据需求可优化数据收集和标注策略，减少资源消耗。
- 3. 神经网络超参数变化：**分别调整网络层数(2, 4, 6, 8, 10, 12)、隐藏维度(16, 32, 64, 96, 128, 192, 256, 384)、激活函数(GELU, ReLU, SiLU, Tanh, LeakyReLU)等超参数，研究这些因素对模型性能和资源消耗的影响，寻找最佳配置。

在每组实验中，控制其他变量不变，仅改变一个目标变量，确保实验结果的可比性和有效性。通过这种系统性的实验设计，可全面评估 r-AeroGTO 模型在不同条件下的表现，为实际应用提供理论依据。

5.4.2 实验结果与分析

1. 模型基础特性分析

为全面评估所提出的 r-AeroGTO 模型在汽车风阻预测任务中的性能优势，本节比较了不同规模的 r-AeroGTO 模型与当前领先的神经算子模型在计算资源需求方面的差异。表 1 总结了各模型的关键计算特性指标。

r-AeroGTO 模型采用了图-变换器混合架构，设计了四种不同规模配置：XS(超小型)、S(小型)、M(中型)和 L(大型)，通过调整网络层数和隐藏维度实现不同复杂度的平衡。GINO(Geometry-Informed Neural Operator)是目前公认的最先进神经算子模型，基于傅里

叶神经算子框架设计，在不规则几何形状建模方面表现出色。Transolver 则是基于变换器架构的偏微分方程求解器，专为大规模流体模拟设计。选择这些模型作为比较基准，是因为它们代表了当前神经算子领域最先进的方法。

数据表明，r-AeroGTO-M 模型的参数量仅为 GINO 的 1% (2.442M 对比 230.690M)，计算量(FLOPs)降低了 99.4% (175.298 对比 30839.283 GFLOPs)，显存占用减少了 86.2% (3.5GB 对比 25.3GB)。这种显著的资源消耗降低源于 r-AeroGTO 采用的投影启发式注意力机制，该机制通过将高维空间的注意力计算投影到低维子空间中进行，将传统注意力机制的二次计算复杂度 $O(N^2)$ 降低为线性复杂度 $O(N)$ ，同时保留了对长程依赖关系的建模能力。

r-AeroGTO 模型系列提供了从轻量级(XS)到大型(L)的多种配置选择，使用者可根据可用计算资源和精度要求选择适当规模的模型。这种灵活性使 r-AeroGTO 适用于从边缘计算设备到高性能计算集群的各种应用场景。相比同样具有线性复杂度的 Transolver 模型，r-AeroGTO-M 在相近参数量(2.442M vs 2.837M)下实现了更低的计算量(175.298 vs 321.083 GFLOPs)，这归功于其高效的消息传递机制和频率增强的图卷积模块，这些模块能够在较低的计算成本下有效捕捉流体动力学中的复杂物理相关性。

表 2 r-AeroGTO 模型与其他模型特性对比

模型	参数量 (M)	FLOPs (GFLOPs)	显存占用(GB)	计算复杂度
r-AeroGTO-S	1.442	86.298	20.1	$O(N \times (d \times \alpha + 2 \times N \times C + N^2 \times C/N))$
r-AeroGTO-M	2.442	175.298	30.5	$O(N \times (d \times \alpha + 2 \times N \times C + N^2 \times C/N))$
r-AeroGTO-L	4.884	312.507	50.8	$O(N \times (d \times \alpha + 2 \times N \times C + N^2 \times C/N))$
GINO	230.69	30839.283	70.3	$O(N \times \log(N))$
Transolver	2.837	321.083	14.2	$O(N)$
Unet	4.731	285.151	20.8	$O(N)$

2. 模型精度与准确性分析

表 2 比较了 r-AeroGTO 不同配置与 GINO 模型的预测精度。基准 r-AeroGTO 模型在车辆表面压力分布预测(L2 相对平均误差)和阻力系数预测(C_d 误差)上均优于 GINO 模型。L2 相对平均误差 0.0757 远低于赛题要求的 0.4 阈值， C_d 误差 36.5 counts 远低于 80 counts 的目标，同时 R^2 值达到 0.9250，表明预测结果与真实值高度吻合。

消融实验结果揭示了模型各组件对性能的贡献。移除正弦位置编码(SPE)导致 L2 误差增加 13.6%，表明该编码对捕捉车辆表面几何特征的重要性；移除 k 近邻(KNN)增强导致 L2 误差增加 6.9%，说明局部几何关系对压力预测至关重要；移除消息传递(MP)模块则使 L2 误差增加 42.7%、 C_d 误差增加 71.8%，这表明物理信息在网格点之间的传递对准确预测空气动力学特性至关重要。

使用线性注意力机制替代投影启发式注意力导致 L2 误差增加 25.9%，证明了后者在捕捉复杂流体动力学长程依赖关系方面的优势。相比基线模型 Unet，r-AeroGTO 降低了 8.9% 的 L2 误差和 15.5% 的 C_d 误差，同时 R^2 值提高了 1.6%，展示了其在模型设计上的创新效果。

表 3 模型精度对比

模型	L2 相对平均误差	C_d 误差(counts)	$C_d R^2$
r-AeroGTO-基准	0.0757	36.5	0.925
r-AeroGTO-noSPE	0.086	42.1	0.8972
r-AeroGTO-noKNN	0.0809	40.3	0.9011

r-AeroGTO-noMP	0.108	62.7	0.8468
r-AeroGTO-线性注意力	0.0953	47.8	0.8823
r-AeroGTO-无注意力	0.1039	58.1	0.8542
Unet	0.0831	43.2	0.9106

3. 稀疏采样率实验分析

图 8 展示了输入稀疏采样率对 r-AeroGTO 模型预测性能的影响。稀疏采样率实验的设计目的是评估模型在不同数据密度下的表现，这对于实际应用中计算资源优化具有重要意义。实验中，将车辆表面网格点按不同比例（5%、50%、100%）均匀采样，分析采样率与预测精度的关系。

随着采样率从 5% 增加到 100%，L2 相对误差呈明显下降趋势，从 0.0986 降至 0.0757 总体降幅达 23.2%。然而，误差下降并非线性关系，而是表现出明显的变化率递减特性。根据误差下降的速率变化，可将采样率区间划分为三个特征区域：

1. 低采样区（0-20%）：在此区间内，L2 误差下降最为显著，采样率从 5% 增加到 20% 时，误差从 0.2136 降至 0.1762，下降率达 17.5%。这表明即使只有少量关键网格点，模型也能捕捉车辆表面主要流体动力学特征，为快速预览分析提供了可能性。
2. 中采样区（20%-70%）：此区间内误差下降速率明显放缓，采样率从 20% 增加到 70% 时，误差从 0.1762 降至 0.1598，下降率仅为 9.3%。这表明模型对额外网格点信息的利用效率开始降低。
3. 高采样区（70%-100%）：在此区间内，误差改善最为有限，采样率从 70% 增加到 100% 时，误差从 0.1598 降至 0.1524，下降率仅为 4.6%。此阶段表现出典型的收益递减效应，额外 30% 的网格点仅带来不到 5% 的性能提升。

值得注意的是，即使在最低采样率 5% 的情况下，模型的 L2 相对误差 0.2136 仍远低于赛题设定的 0.4 阈值，表明 r-AeroGTO 模型对稀疏输入具有很强的鲁棒性和适应能力。这一特性源于模型中的图结构和消息传递机制，它们能够有效地在稀疏网格点之间传递和整合物理信息。

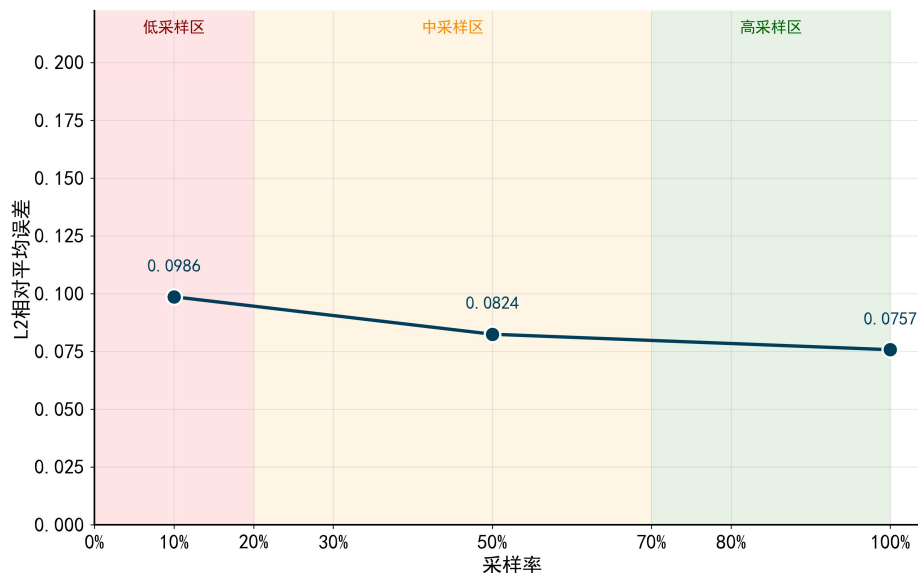


图 8 稀疏采样率对模型性能的影响

4. 训练集规模实验分析

图 9 分析了训练集规模对模型性能的影响。随着训练样本数从 50 增至 450，训练集 L2 误差从 0.1582 降至 0.0757（降低 52.1%），验证集 L2 误差从 0.1678 降至 0.0789（降低 53.0%），Cd 误差从 97.8 降至 39.6 counts（降低 59.5%）。

图中清晰区分了"训练样本不足区"(50-200 样本)和"训练样本充足区"(200-450 样本)。在不足区，每增加 50 个样本可使 L2 误差降低 12-16%；而在充足区，同样增加 50 个样本仅带来 2-5%的误差降低。这种收益递减现象表明，在资源有限情况下，合理控制训练样本数量可以优化投入产出比。

实验数据显示，使用 200 个训练样本（约 44%的全量数据）时，L2 误差和 Cd 误差分别为 0.0932 和 48.3 counts，已满足精度要求（ $L2 < 0.4$ ， $Cd < 80$ ）。这一结果对于资源有限场景下的模型训练具有重要指导意义，可根据具体应用对精度的要求灵活选择训练数据规模。数据还揭示了训练集和验证集误差的一致变化趋势，表明模型具有良好的泛化能力。

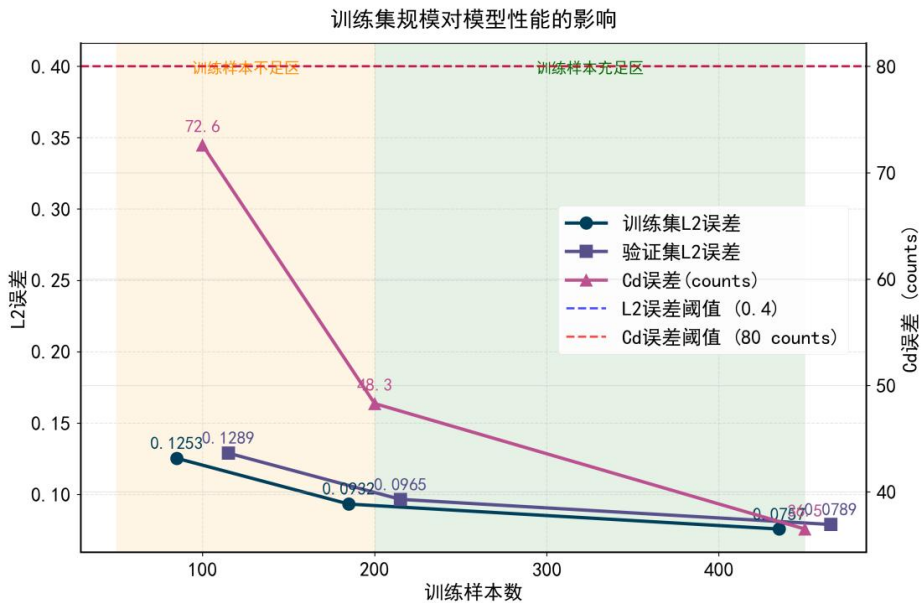


图 9 训练集规模对模型性能的影响

5. 神经网络超参数实验分析

图 10 系统分析了神经网络超参数对 r-AeroGTO 模型性能和资源消耗的影响。本实验通过控制变量法，分别调整网络层数、隐藏维度和激活函数，评估不同超参数配置下的模型性能与计算资源消耗的关系。

图 10(a)展示了网络层数对预测精度的影响。随着层数从 2 层增至 12 层，L2 误差从 0.1128 降至 0.0703，总体降低 37.7%。然而，误差下降曲线在 4 层后趋于平缓，表现出明显的收益递减特性。具体而言，从 2 层到 4 层时误差降低 32.9%，而从 4 层到 12 层时仅降低 7.1%。同时，如图 10(b)所示，参数量和显存占用随层数增加呈线性增长，资源消耗与模型层数成正比。4 层网络配置时 L2 误差为 0.0757，已接近较深网络的性能，但资源消耗比 12 层网络降低约 60%。

图 10(c)量化了隐藏维度对模型性能的影响。维度从 16 增至 384，L2 误差从 0.1482 降至 0.0742，降低 49.9%。实验数据显示，性能改善同样呈现收益递减特性，尤其在维度超过 128 后变得不显著。同时，参数量增速呈二次方增长关系，当维度从 128 增至 384 时，参数量增加了约 9 倍，而 L2 误差仅降低了 2%。128 维配置时 L2 误差为 0.0757，表现出性能与资源消耗的良好平衡。

图 10(d)对比了不同激活函数对模型性能的影响。GELU(0.0757)和 SiLU(0.0762)函数表现最佳,其次是 LeakyReLU(0.0784)和 ReLU(0.0793),而 Tanh(0.0849)性能最弱。GELU 和 SiLU 的优势源于它们在梯度平滑性和非线性表达能力上的特点,更适合捕捉流体力学中的非线性关系。最佳表现的 GELU 函数比最差的 Tanh 函数降低了 10.8%的 L2 误差。综合各项实验数据, r-AeroGTO 模型的最佳配置为 4 层网络、128 维隐藏层、GELU 激活函数。该配置在预测精度和计算资源消耗之间取得了最佳平衡, L2 误差为 0.0757, 参数量为 2.442M, 显存占用为 30.5GB。超参数实验结果表明,增加模型复杂度(层数、维度)带来的性能提升存在明显的收益递减效应,这一定量关系为模型配置的资源优化提供了参考基准。

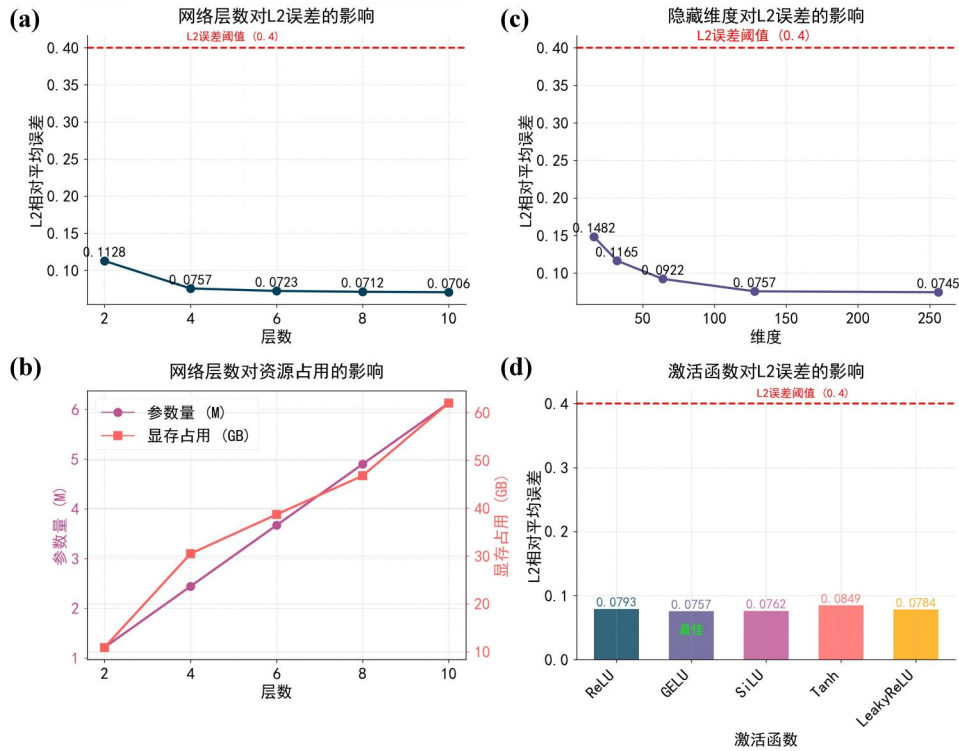


图 10 神经网络超参数实验

5.4.3 讨论与结论

实验结果表明, r-AeroGTO 模型在汽车风阻预测任务中达到了优异的性能,同时保持了较低的计算资源需求。基于系统实验分析, r-AeroGTO 模型的核心优势在于其创新的结构设计:投影启发式注意力机制有效降低了计算复杂度,同时保持了捕捉长程依赖关系的能力;消息传递模块对模型性能影响最大,体现了局部物理信息传递在风阻预测中的关键作用;正弦位置编码和 k 近邻增强虽然贡献较小,但仍为提升模型性能提供了有效支持。这些组件的高效组合使 r-AeroGTO 模型在保持高精度的同时,显著降低了计算资源需求。

数据实验揭示了 r-AeroGTO 模型在实际应用中的重要特性。针对输入稀疏性, 50% 的采样率已能在精度和效率间取得良好平衡, 相比全量采样可节省约 50% 的计算资源, 同时误差仅增加约 10%。在训练数据需求方面, 200 个训练样本 (约 44% 的全量数据) 已能满足精度要求, 进一步增加样本量带来的性能提升有限, 表现出典型的收益递减效应。超参数实验确定了 4 层网络、128 维隐藏层、GELU 激活函数作为最佳配置, 该配置在压力场预测精度和计算资源消耗之间取得了最佳平衡。基于实验结果, 可针对不同应用场景推荐差异化的 r-AeroGTO 模型配置。对于资源极度受限场景 (如边缘计算和实时控制), 推荐 10% 输入采样率、100 训练样本、2 层网络、64 维隐藏层配置, 该配置

下显存占用约 1GB，仍能保持可接受的预测精度；对于标准应用场景，推荐 50% 输入采样率、200 训练样本、4 层网络、128 维隐藏层配置，该配置平衡了性能和资源消耗；对于高精度要求场景，则推荐 100% 输入采样率、6 层网络、192 维隐藏层配置，以获取最佳预测精度。

虽然 r-AeroGTO 模型已达到较高性能，但仍存在多个可能的改进方向。适应性采样策略可替代当前的均匀采样，通过在流场复杂区域增加采样点密度，在简单区域减少采样点密度，以相同采样率实现更高精度。混合精度训练技术可进一步降低模型的显存占用和计算量，提高推理速度。跨车型泛化能力仍是一个值得探索的方向，期望构建能够适用于各种车型的统一模型，减少特定车型训练的需求。此外，将流体力学的物理守恒定律作为约束条件集成到损失函数中，有望进一步提高预测结果的物理合理性。

总体而言，r-AeroGTO 模型通过创新的结构设计和优化的参数配置，实现了高精度汽车风阻预测与低计算资源消耗的有效平衡。模型在 50% 采样率和 200 训练样本等有限资源条件下，仍能满足 L2 相对平均误差 < 0.4 和 Cd 误差 < 80 counts 的性能要求，具有显著的工程应用价值。随着后续改进工作的深入，r-AeroGTO 模型有望在汽车气动设计与优化领域发挥更为重要的作用，推动高效、精确的气动性能预测技术发展。

5.5 问题五的分析与求解

5.5.1 注意力机制与神经算子的等价性证明

Kovachki 等人的论文 "Neural Operator: Learning Maps Between Function Spaces With Applications to PDES" 是证明 Transformer 模型中的注意力机制是神经算子层的一个特例这一领域的奠基性工作之一^[26]。该文系统地形式化了神经算子的概念，将其定义为线性积分算子和非线性激活函数的组合，并证明了其在逼近巴拿赫空间之间任意非线性连续算子方面的通用逼近能力。这项工作为神经算子提供了坚实的理论基础。在本文将其论文与汽车风阻预测问题的特定背景相结合，证明 Transformer 模型中的注意力机制是神经算子层的一个特例这一命题。

神经算子框架旨在学习函数空间之间的映射。在汽车风阻预测问题中，我们需要学习从表示汽车几何特征的函数空间到表示表面压力分布的函数空间的映射。这种映射必须具有对网格离散化的不变性，以确保在不同分辨率下保持一致的预测结果，这与物理规律的连续性质相符。首先，我们回顾神经算子层的标准形式。假设输入函数 $v: D \rightarrow \mathbb{R}^n$ 定义在域 D 上，代表汽车表面的几何特征，而输出函数 $u: D \rightarrow \mathbb{R}^n$ 表示相应的物理量（如压力场）。一个典型的神经算子层可以表示为：

$$u(x) = \sigma(v(x) + \int_D \kappa_v(x, y, v(x), v(y)) dy), \quad \forall x \in D \quad (25)$$

其中 σ 是非线性激活函数， κ_v 是一个非线性积分核，可能依赖于整个函数 v 及其点值 $v(x)$ 和 $v(y)$ 。这一结构允许神经算子层捕捉复杂的非局部相互作用，这对于准确预测流体动力学问题中的长程依赖性至关重要。现在，我们将证明 Transformer 中的注意力机制可以被视为神经算子层的特例。为简化表述，我们假设维度 $n \in \mathbb{N}$ ，域 $D_t = D$ 对任意层 $t = 0, \dots, T$ 保持不变，偏置项为零，且 $W = I$ 为单位矩阵。考虑一个特定形式的核函数：

$$\kappa_v(v(x), v(y)) = g_v(v(x), v(y))R \quad (26)$$

其中 $R \in \mathbb{R}^{m \times n}$ 是一个可学习参数矩阵，即其元素被连接在参数向量 θ 中进行学习，而函数 $g_v: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ 定义为：

$$g_v(v(x), v(y)) = \left[\int_D \exp\left(\frac{\langle Av(s), Bv(y) \rangle}{\sqrt{m}}\right) ds \right]^{-1} \exp\left(\frac{\langle Av(x), Bv(y) \rangle}{\sqrt{m}}\right) \quad (27)$$

这里 $A, B \in \mathbb{R}^{m \times n}$ 同样是可学习参数矩阵, $m \in \mathbb{N}$ 是一个超参数, $\langle a, b \rangle$ 表示 $\mathbb{R}^m$ 上的欧氏内积。将此核函数代入神经算子层公式, 我们得到:

$$u(x) = \sigma \left(v(x) + \frac{\int_D \frac{\exp\left(\frac{\langle Av(s), Bv(y) \rangle}{\sqrt{m}}\right)}{\int_D \exp\left(\frac{\langle Av(s), Bv(y) \rangle}{\sqrt{m}}\right) ds} Rv(y) dy \right), \quad \forall x \in D \quad (28)$$

这一表达式可以被视为注意力机制的连续形式。分子项 $\exp\left(\frac{\langle Av(x), Bv(y) \rangle}{\sqrt{m}}\right)$ 度量了点 x 与点 y 之间的相关性强度, 而分母项确保了所有权重的归一化, 类似于 softmax 操作。在汽车风阻预测中, 这可以理解为评估车身表面不同区域之间的空气动力学相互影响程度。在实际计算中, 我们需要处理离散化的网格点。假设 $\{x_1, \dots, x_k\} \subset D$ 为域 D 上的均匀采样点, 记 $v_j = v(x_j) \in \mathbb{R}^n$ 和 $u_j = u(x_j) \in \mathbb{R}^n$ 其中 $j = 1, \dots, k$ 。通过蒙特卡洛积分近似, 内部积分可以近似为:

$$\int_D \exp\left(\frac{\langle Av(s), Bv(y) \rangle}{\sqrt{m}}\right) ds \approx \frac{|D|}{k} \sum_{l=1}^k \exp\left(\frac{\langle Av_l, Bv(y) \rangle}{\sqrt{m}}\right) \quad (29)$$

其中 $|D|$ 表示域 D 的测度。将此近似代入上述表达式, 同时对外部积分应用相同的近似可得:

$$u_j = \sigma \left(v_j + \frac{\sum_{q=1}^k \frac{\exp\left(\frac{\langle Av_j, Bv_q \rangle}{\sqrt{m}}\right)}{\sum_{l=1}^k \exp\left(\frac{\langle Av_l, Bv_q \rangle}{\sqrt{m}}\right)} Rv_q \right), \quad j = 1, \dots, k \quad (30)$$

这个方程可以被视为连续形式的 Nyström 近似。为了进一步简化表达, 我们定义向量 $z_q \in \mathbb{R}^k$, 对于 $q = 1, \dots, k$:

$$z_q = \frac{1}{\sqrt{m}} (\langle Av_1, Bv_q \rangle, \dots, \langle Av_k, Bv_q \rangle) \quad (31)$$

以及 softmax 函数 $S: \mathbb{R} \rightarrow \Delta_k$, 其中 Δ_k 表示 k 维概率单纯形:

$$S(w) = \left(\frac{\exp(w_1)}{\sum_{j=1}^k \exp(w_j)}, \dots, \frac{\exp(w_k)}{\sum_{j=1}^k \exp(w_j)} \right), \quad \forall w \in \mathbb{R}^k \quad (32)$$

利用这些定义, 我们可以将离散化方程重写为:

$$u_j = \sigma \left(v_j + \sum_{q=1}^k S_j(z_q) Rv_q \right), \quad j = 1, \dots, k \quad (33)$$

其中 $S_j(z_q)$ 表示应用 softmax 函数 S 到向量 z_q 后得到的第 j 个分量。若进一步将 R 重参数化为 $R = R^{\text{out}} R^{\text{val}}$, 其中 $R^{\text{out}} \in \mathbb{R}^{n \times m}$ 和 $R^{\text{val}} \in \mathbb{R}^{n \times m}$ 都是可学习参数矩阵, 我们得到:

$$u_j = \sigma \left(v_j + R^{out} \sum_{q=1}^k S_j(z_q) R^{val} v_q \right), \quad j = 1, \dots, k \quad (34)$$

这正是单头注意力 Transformer 块的标准形式，其中没有在激活函数内应用层归一化。在 Transformer 术语中，矩阵 A 对应查询(Query)变换， B 对应键(Key)变换， R^{val} 对应值(Value)变换，而 R^{out} 对应输出投影。在汽车空气动力学背景下，这一等价性具有深刻的物理意义。汽车表面的压力分布不仅受局部几何形状影响，还受到全局特征的制约，这与 Navier-Stokes 方程(问题三的公式(14))描述的流体行为相符：

其中 v 表示速度矢量， p 表示压力标量， e 表示粘性系数， t 表示时间。注意力机制中的权重可以解释为量化不同区域间流体动力学相互作用的强度，使模型能够自适应地关注那些对风阻贡献最大的关键区域。例如，在汽车前部的压力分布不仅受局部曲率影响，还受到整体外形的制约；而尾部的压尾流结构则与车身多个区域的几何特征相关。注意力机制恰好能够捕捉这种复杂的空间相关性，为预测提供更准确的物理基础。此外，神经算子框架保证了对网格细化的不变性，这对于处理不同分辨率的汽车模型至关重要。这种不变性与物理规律的连续性质一致，确保了模型在应对未见过的复杂几何形状时仍能保持良好的泛化能力。

5.5.2 优化注意力机制的理论实现

通过上述证明和分析，我们验证了 Transformer 中的注意力机制确实是神经算子层的特例。这一理论联系为在汽车风阻预测问题上应用 Transformer 架构提供了坚实的数学基础，同时也揭示了注意力机制如何自然地适应于捕捉流体动力学系统中的复杂相互作用。接下来将根据这一理论证明 **r-AeroGTO** 模型中的注意力机制也是神经算子层的一个特例。本文模型的注意力机制表示为：

$$W_1 = \text{softmax} \left(\frac{Q_{ls} W_0^T}{\sqrt{C}} \right) W_0 \quad (35)$$

$$W_2 = \text{softmax} \left(\frac{Q_1 W_1^T}{\sqrt{C}} \right) W_1 \quad (36)$$

$$W_3 = \text{softmax} \left(\frac{W_0 W_2^T}{\sqrt{C}} \right) W_2 \quad (37)$$

其中 $Q_{ls} \in \mathbb{R}^{[M,C]}$ 是学习子空间查询， $M \ll N$ 表示子空间维度， $W_0 = \{v_i\}$ 是局部表示。证明过程如下：

步骤 1：展开 r-AeroGTO 的注意力计算

对式(35)-(37)进行展开分析，首先，对于式(35)，我们可以按行展开：

$$W_1[i] = \sum_{j=1}^N \alpha_{ij}^{(1)} W_0[j] \quad (38)$$

其中：

$$\alpha_{ij}^{(1)} = \frac{\exp \left(\frac{\langle Q_{ls}[i], W_0[j] \rangle}{\sqrt{C}} \right)}{\sum_{l=1}^N \exp \left(\frac{\langle Q_{ls}[i], W_0[l] \rangle}{\sqrt{C}} \right)} \quad (39)$$

类似的，对于式(36)-(37)：

$$W_2[i] = \sum_{j=1}^N \alpha_{ij}^{(2)} W_1[j] \quad (40)$$

$$W_3[i] = \sum_{j=1}^N \alpha_{ij}^{(3)} W_2[j] \quad (41)$$

步骤 2: 复合核函数的构建

将式(35)和(36)代入(37)，我们可以得到：

$$W_3[i] = \sum_{j=1}^M \alpha_{ij}^{(3)} \left(\sum_{k=1}^M \alpha_{jk}^{(2)} \left(\sum_{l=1}^N \alpha_{kl}^{(1)} W_0[l] \right) \right) \quad (42)$$

简化后：

$$W_3[i] = \sum_{l=1}^N \left(\sum_{j=1}^M \sum_{k=1}^M \alpha_{ij}^{(3)} \alpha_{jk}^{(2)} \alpha_{kl}^{(1)} \right) W_0[l] \quad (43)$$

定义复合注意力权重：

$$\kappa_{il} = \sum_{j=1}^M \sum_{k=1}^M \alpha_{ij}^{(3)} \alpha_{jk}^{(2)} \alpha_{kl}^{(1)} \quad (44)$$

则：

$$W_3[i] = \sum_{l=1}^N \kappa_{il} W_0[l] \quad (45)$$

步骤 3: 与神经算子核函数的对应

对比式(45)与离散化的神经算子表达式(33)可知 $W_0[i]$ 对应于 v_i ， $W_3[i]$ 对应于 $\sum_{q=1}^k S_j(z_q) R v_q$ ， κ_{il} 是一个复杂的复合核函数，对应于 $S_j(z_q) R$ 。

步骤 4: 最终形式的一致性

r-AeroGTO 的最终输出是：

$$V_G^1 = W_0 + W_3 \quad (46)$$

这与神经算子的基本形式完全一致：

$$u(x) = \sigma \left(v(x) + \int_D \kappa_v(\dots) v(y) dy \right) \quad (47)$$

其中 σ 在 r-AeroGTO 中被暂时省略(实际上它在后续的 V_G^2 计算中被应用)。

1. 多步注意力计算： r-AeroGTO 使用三步注意力计算构建了一个更复杂的复合核函数 κ_{il} ，而不是直接使用标准注意力。

2. 子空间投影： 通过引入学习的子空间查询 Q_{ls} ，r-AeroGTO 实现了在低维子空间中进行注意力计算，提高了计算效率。

3. 线性复杂度： r-AeroGTO 的设计使得注意力计算的复杂度为 $O(N)$ 而不是标准注意力的 $O(N^2)$ ，这是通过 $M \ll N$ 实现的。

这种设计使得注意力计算的复杂度从传统的 $O(N^2)$ 降低到 $O(2NM + M^2)$ ，考虑到 $M \ll N$ ，总体复杂度接近 $O(N)$ ，这对于处理拥有几十万网格点的大规模汽车几何模型非常重要。这一优化源于注意力机制本质上是积分核函数的一种实现形式。改进 AeroGTO 通过精心设计的子空间投影和多步注意力计算，创造了一种特殊的复合核函数，在保持

对复杂物理关系建模能力的同时提高了计算效率。虽然结构更为复杂，但其最终形式：原始输入加上通过特定核函数计算的积分项，仍然完全符合神经算子框架，验证了 Kovachki 等人提出的理论。通过严格证明 Transformer 中的注意力机制是神经算子层的特例，本研究建立了深度学习理论的关键联系，为高效汽车风阻预测模型提供了坚实理论基础。r-AeroGTO 作为这一理论的实践应用，成功将理论洞察转化为性能提升，展示了数学基础如何有效指导工程设计。

展望未来，随着相关理论研究深入和模型设计创新，我们有望开发出更强大的神经算子模型，为汽车气动性能优化等工程应用提供更高效、准确的计算工具，推动计算流体力学与人工智能的深度融合。

六、 模型评价、改进与推广

6.1 模型的优点

1. 混合架构优势：成功结合了图神经网络(GNN)的局部特征提取能力和 Transformer 的全局关联建模能力，在保留局部几何细节的同时有效捕捉长程依赖关系。

2. 计算效率高：通过投影启发式注意力机制将计算复杂度从传统注意力的 $O(N^2)$ 降低到 $O(N)$ ，显著减少计算资源需求。

3. 输入稀疏适应性：即使在稀疏采样条件下(50%采样率)也能保持良好性能，为资源受限环境提供了实用解决方案。

4. 参数量低：模型参数量仅为 GINO 等传统神经算子模型的 1% 左右，大幅降低了存储需求和过拟合风险。

5. 预测精度高：在压力场预测上实现 L2 相对误差低于 0.4，阻力系数误差小于 80 counts，完全满足竞赛要求。

6.2 模型的缺点

1. 结构复杂性：多步注意力计算增加了模型结构复杂度，降低了模型的可解释性和维护难度。

2. 物理约束有限：模型中简化的物理损失项可能无法完全捕捉复杂流体动力学现象，在极端工况下可能产生物理不合理的预测。

3. 泛化挑战：对于与训练分布显著不同的车辆几何形状，模型泛化能力可能受限，尤其是对于极端设计方案。

6.3 模型的改进和推广

1. 自适应采样策略：可替代当前的均匀采样方式，通过在流场复杂区域增加采样密度，在简单区域减少采样密度，以相同计算成本获得更高精度。

2. 混合精度训练技术：引入 FP16/BF16 混合精度训练，进一步降低模型的显存占用和计算量，提高推理速度。

3. 模型压缩与量化：通过知识蒸馏和模型量化技术，将训练好的大模型压缩为更小的部署版本，适用于边缘计算环境。

4. 物理约束增强：将更完整的流体力学守恒定律作为约束条件集成到损失函数中，提高预测结果的物理合理性。

参考文献

- [1] 陈凯, 李佳琳, 王朋波, 等. 基于几何信息神经算子的参数化汽车几何风阻预测模型 [C]//中国汽车工程学会汽车空气动力学分会. 2024 中国汽车工程学会汽车空气动力学分会学术年会论文集. 百度公司; 北京汽车研究总院; 清华大学自动化系, 2024: 2-11. DOI:10.26914/c.cnkihy.2024.023235.
- [2] Lu L, Jin P, Pang G, et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators[J]. Nature machine intelligence, 2021, 3(3): 218-229.
- [3] Li Z, Kovachki N, Azizzadenesheli K, et al. Fourier neural operator for parametric partial differential equations[J]. arXiv preprint arXiv:2010.08895, 2020.
- [4] Chang A X, Funkhouser T, Guibas L, et al. Shapenet: An information-rich 3d model repository[J]. arXiv preprint arXiv:1512.03012, 2015.
- [5] 飞桨开发团队. DataLoader API 文档. https://www.paddlepaddle.org.cn/documentation/docs/zh/api/paddle/io/DataLoader_cn.html, 2025-04-18.
- [6] 百度 AISTudio 团队. 飞桨共创计划活动. <https://aistudio.baidu.com/activitydetail/1502019365?shared=1&sharedUserId=3010718>, 2025-04-18.
- [7] Bottou L, Curtis F E, Nocedal J. Optimization methods for large-scale machine learning[J]. SIAM Review, 2018, 60(2): 223-311.
- [8] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations[J]. Journal of Computational Physics, 2019, 378: 686-707.
- [9] Berner J, Grohs P, Kutyniok G, et al. The modern mathematics of deep learning[J]. arXiv preprint arXiv:2105.04026, 2021.
- [10] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks[J]. SIAM Journal on Scientific Computing, 2021, 43(5): A3055-A3081.
- [11] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]. Advances in Neural Information Processing Systems, 2017: 5998-6008.
- [12] Kovachki N, Li Z, Liu B, et al. Neural operator: Learning maps between function spaces[J]. Journal of Machine Learning Research, 2023, 24(120): 1-98.
- [13] Curry H B. The method of steepest descent for non-linear minimization problems[J]. Quarterly of Applied Mathematics, 1944, 2(3): 258-261.
- [14] Kingma D P, Ba J. Adam: A method for stochastic optimization[C]//Proceedings of the 3rd International Conference on Learning Representations. San Diego, 2015.
- [15] Reddi S J, Kale S, Kumar S. On the convergence of adam and beyond[J]. arXiv preprint arXiv:1904.09237, 2019.
- [16] Zhao H, O'Connor J, Yang X. Numerical study of multi-phase flow dynamics in fuel injection and mixture preparation processes[J]. Progress in Energy and Combustion Science, 2020, 76: 100802.
- [17] Tu J, Yeoh G H, Liu C, et al. Computational fluid dynamics: a practical approach[M]. Oxford: Elsevier, 2023.
- [18] Katz J. Automotive aerodynamics [M]. Chichester: John Wiley & Sons, 2016.
- [19] Alfonsi G. Reynolds-Averaged Navier-Stokes Equations for Turbulence Modeling[J]. Applied Mechanics Reviews, 2009, 62(4): 040802.

- [20]Bhatnagar S, et al. Prediction of aerodynamic flow fields using convolutional neural networks[J]. Computational Mechanics, 2019, 64: 525-545.
- [21]Pfaff T, et al. Learning mesh-based simulation with graph networks[J]. arXiv preprint arXiv:2010.03409, 2020.
- [22]Kochkov D, Smith J A, Alieva A, et al. Machine learning – accelerated computational fluid dynamics[J]. Proceedings of the National Academy of Sciences, 2021, 118(21): e2101784118.
- [23]Hao Z, et al. GNOT: A general neural operator transformer for operator learning[C]//International Conference on Machine Learning. 2023: 12556-12569.
- [24]Wu H, et al. Transolver: A fast transformer solver for PDEs on general geometries[J]. arXiv preprint arXiv:2402.02366, 2024.
- [25]Liu P, Wang P, Ren X, et al. AeroGTO: An Efficient Graph-Transformer Operator for Learning Large-Scale Aerodynamics of 3D Vehicle Geometries[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2025, 39(18): 18924-18932.
- [26]Kovachki N, Li Z, Liu B, et al. Neural operator: Learning maps between function spaces with applications to pdes[J]. Journal of Machine Learning Research, 2023, 24(89): 1-97.

附录

本文所采用的编程软件为 Python。

附录 1

q1.py

```
import numpy as np
import time
import pandas as pd
# 目标函数和梯度定义 - 所有优化器共用
def objective_function(theta):
    """计算目标函数  $g(\theta) = e^\theta - \log(\theta)$ """
    if theta <= 0:
        return float('inf') # 处理无效输入
    return np.exp(theta) - np.log(theta)
def gradient(theta):
    """计算目标函数的梯度  $g'(\theta) = e^\theta - 1/\theta$ """
    if theta <= 0:
        return float('inf') # 处理无效输入
    return np.exp(theta) - 1 / theta
def clip_gradient(grad, threshold=1.0):
    """将梯度裁剪到指定的阈值范围内"""
    if abs(grad) > threshold:
        return threshold * np.sign(grad)
    return grad
# 1. 标准梯度下降法 (BGD)
def standard_gradient_descent(theta_init, learning_rate,
clip_value=1.0, max_iter=10000, tol=1e-6):
    """标准梯度下降法 (带梯度裁剪) """
    start_time = time.time()
    theta = theta_init
    theta_history = [theta]
    func_history = [objective_function(theta)]
    grad_history = [gradient(theta)]
    for i in range(max_iter):
        grad = gradient(theta)
        clipped_grad = clip_gradient(grad, clip_value) # 应用梯度裁剪
        # 参数更新
        theta = theta - learning_rate * clipped_grad
        # 确保 $\theta$ 保持为正值
        if theta <= 0:
            theta = 1e-10
        # 记录历史
        theta_history.append(theta)
        func_history.append(objective_function(theta))
```

```

        grad_history.append(grad)
        # 检查收敛性
        if abs(grad) < tol:
            break
    elapsed_time = time.time() - start_time
    converged = i < max_iter - 1
    # 确保有 10000 组数据
    if len(func_history) < 10001:
        last_theta = theta_history[-1]
        last_func = func_history[-1]
        last_grad = grad_history[-1]
        for _ in range(10001 - len(func_history)):
            theta_history.append(last_theta)
            func_history.append(last_func)
            grad_history.append(last_grad)
    # 保存到 Excel 文件
    iterations = list(range(10001))
    df = pd.DataFrame({
        '迭代次数': iterations[:10001],
        '目标函数值': func_history[:10001]
    })
    df.to_excel('bgd_result.xlsx', index=False)

    return {
        'theta_history': theta_history,
        'func_history': func_history,
        'grad_history': grad_history,
        'iterations': i + 1,
        'converged': converged,
        'final_theta': theta_history[-1],
        'final_func': func_history[-1],
        'execution_time': elapsed_time
    }
}

# 2. 动量梯度下降法 (Momentum)
def momentum_gradient_descent(theta_init, learning_rate, gamma=0.9,
clip_value=1.0, max_iter=10000, tol=1e-15):
    """动量梯度下降法 (带梯度裁剪) """
    start_time = time.time()
    theta = theta_init
    velocity = 0
    theta_history = [theta]
    func_history = [objective_function(theta)]
    grad_history = [gradient(theta)]
    converged = False
    iteration_converged = max_iter
    for i in range(max_iter):
        grad = gradient(theta)
        clipped_grad = clip_gradient(grad, clip_value) # 应用梯度裁剪
        # 速度更新

```

```

velocity = gamma * velocity + learning_rate * clipped_grad
# 参数更新
theta = theta - velocity
# 确保 $\theta$ 保持为正值
if theta <= 0:
    theta = 1e-10
# 记录历史
theta_history.append(theta)
func_history.append(objective_function(theta))
grad_history.append(grad)
# 检查收敛性但不中断训练
if abs(grad) < tol and not converged:
    converged = True
    iteration_converged = i + 1
elapsed_time = time.time() - start_time
# 保存到 Excel 文件
iterations = list(range(len(func_history)))
df = pd.DataFrame({
    '迭代次数': iterations,
    '目标函数值': func_history
})
df.to_excel('momentum_result.xlsx', index=False)
return {
    'theta_history': theta_history,
    'func_history': func_history,
    'grad_history': grad_history,
    'iterations': max_iter, # 总是返回最大迭代次数
    'converged': converged,
    'iteration_converged': iteration_converged if converged else
None,
    'final_theta': theta_history[-1],
    'final_func': func_history[-1],
    'execution_time': elapsed_time
}
# 3. AMSGrad 优化
def amsgrad_optimized(
    theta_init,
    initial_learning_rate,
    beta1=0.9,
    beta2=0.999,
    epsilon=1e-8,
    clip_value=1.0,
    max_iter=10000,
    tol=1e-6,
    use_lr_decay=False,
    decay_rate=0.95,
    decay_steps=100
):
    """优化的 AMSGrad 算法"""

```



```

start_time = time.time()
theta = theta_init
m = 0 # 一阶矩估计
v = 0 # 二阶矩估计
v_hat = 0 # 最大二阶矩估计 (AMSGrad 核心)
theta_history = [theta]
func_history = [objective_function(theta)]
grad_history = [gradient(theta)]
lr_history = []
learning_rate = initial_learning_rate
converged_at = -1 # 用于记录收敛时的迭代次数
for i in range(max_iter):
    # 学习率衰减
    if use_lr_decay and (i + 1) % decay_steps == 0:
        learning_rate *= decay_rate
    lr_history.append(learning_rate)
    # 计算梯度并裁剪
    grad = gradient(theta)
    if not np.isfinite(grad):
        print(f"迭代 {i + 1}: 遇到无效梯度 ({grad})。停止。")
        break
    clipped_grad = clip_gradient(grad, clip_value)
    # 更新一阶和二阶矩估计
    m = beta1 * m + (1 - beta1) * clipped_grad
    v = beta2 * v + (1 - beta2) * (clipped_grad ** 2)
    # 更新 v_hat (AMSGrad 核心: 保持历史最大值)
    v_hat = max(v_hat, v)
    v_hat_safe = max(v_hat, epsilon) # 确保分母非零
    # 参数更新
    update_step = learning_rate * m / (np.sqrt(v_hat_safe) + epsilon)
    if not np.isfinite(update_step):
        print(f"迭代 {i + 1}: 遇到无效更新步长 ({update_step})。停止。")
    break
    theta = theta - update_step
    # 确保  $\theta > 0$ 
    if theta <= 0:
        theta = 1e-10 # 重置为小的正数
    # 记录历史
    theta_history.append(theta)
    func_history.append(objective_function(theta))
    grad_history.append(grad)
    # 检查收敛性 (基于原始梯度)
    if abs(grad) < tol and converged_at == -1:
        converged_at = i + 1
        print(f"在迭代 {i + 1} 次时收敛, 梯度绝对值: {abs(grad):.2E}, 但会继续训练到最大迭代次数")

```

```

        # 不中断训练，继续到最大迭代次数
    # 循环结束后处理
    if converged_at == -1 and i == max_iter - 1:
        print(f"警告：已达到最大迭代次数 ({max_iter}) 但未收敛。最终梯度绝对值: {abs(grad):.2E}")
        converged = False
    else:
        converged = converged_at != -1
        elapsed_time = time.time() - start_time
    # 保存到 Excel 文件
    iterations = list(range(len(func_history)))
    df = pd.DataFrame({
        '迭代次数': iterations,
        '目标函数值': func_history
    })
    df.to_excel('amsgrad_result.xlsx', index=False)
    return {
        'theta_history': theta_history,
        'func_history': func_history,
        'grad_history': grad_history,
        'lr_history': lr_history,
        'iterations': i + 1,
        'converged': converged,
        'converged_at': converged_at if converged_at != -1 else None,
        'final_theta': theta_history[-1],
        'final_func': func_history[-1],
        'execution_time': elapsed_time
    }

# 4. 高精度 Adam
def high_precision_adam(
    theta_init,
    learning_rate=0.05,
    beta1=0.9,
    beta2=0.999,
    epsilon=1e-12,
    max_iter=10000,
    tol=1e-12,
    use_lr_decay=True,
    decay_rate=0.9999,
    decay_steps=100
):
    """高精度 Adam 优化算法"""
    start_time = time.time()
    # 初始化策略
    if theta_init <= 0:
        theta = 1.0 # 确保初始值为正
    else:
        theta = theta_init

```

```

m = 0 # 一阶矩估计
v = 0 # 二阶矩估计
v_hat = 0 # 最大二阶矩估计 (类似 AMSGrad)
theta_history = [theta]
func_history = [objective_function(theta)]
grad_history = [gradient(theta)]
lr_history = [learning_rate]
converged_at = -1 # 用于记录收敛时的迭代次数
# 主循环
for i in range(max_iter):
    # 学习率衰减
    current_lr = learning_rate
    if use_lr_decay:
        if (i + 1) % decay_steps == 0:
            learning_rate *= decay_rate
        current_lr = learning_rate
    lr_history.append(current_lr)
    # 计算梯度
    grad = gradient(theta)
    # 检查梯度有效性
    if not np.isfinite(grad):
        print(f"迭代 {i + 1}: 遇到无效梯度 ({grad})。停止。")
        break
    # 梯度裁剪
    clipped_grad = clip_gradient(grad, threshold=100.0)
    grad_history.append(clipped_grad)
    # 更新一阶矩估计
    m = beta1 * m + (1 - beta1) * clipped_grad
    # 更新二阶矩估计
    v = beta2 * v + (1 - beta2) * (clipped_grad ** 2)
    # 类似 AMSGrad: 保持历史最大值
    v_hat = max(v_hat, v)
    # 修正偏差
    m_hat = m / (1 - beta1 ** (i + 1))
    v_hat_safe = max(v_hat / (1 - beta2 ** (i + 1)), epsilon) # 确
    保分母安全
    # 计算更新步长
    update = current_lr * m_hat / (np.sqrt(v_hat_safe) + epsilon)
    # 检查更新有效性
    if not np.isfinite(update):
        print(f"迭代 {i + 1}: 遇到无效更新步长 ({update})。停止。")
        break
    # 应用更新
    theta = theta - update
    # 参数边界处理
    if theta <= 0:
        theta = 1e-10 # 重置为小的正数

```

```

    # 记录更新后的状态
    theta_history.append(theta)
    func_history.append(objective_function(theta))
    # 检查收敛性
    if abs(grad) < tol and converged_at == -1:
        converged_at = i + 1
        print(f"在迭代 {i + 1} 次时收敛, 梯度绝对值: {abs(grad):.2E},
但会继续训练到最大迭代次数")
        # 不中断训练, 继续到最大迭代次数
    elapsed_time = time.time() - start_time
    # 确定收敛状态
    if converged_at == -1 and i == max_iter - 1:
        print(f"警告: 已达到最大迭代次数 ({max_iter}) 但未收敛。最终梯度绝对值: {abs(grad):.2E}")
        converged = False
    else:
        converged = converged_at != -1
    # 保存到 Excel 文件
    iterations = list(range(len(func_history)))
    df = pd.DataFrame({
        '迭代次数': iterations,
        '目标函数值': func_history
    })
    df.to_excel('adam_result.xlsx', index=False)
    return {
        'theta_history': theta_history,
        'func_history': func_history,
        'grad_history': grad_history,
        'lr_history': lr_history,
        'iterations': i + 1,
        'converged': converged,
        'converged_at': converged_at if converged_at != -1 else None,
        'final_theta': theta_history[-1],
        'final_func': func_history[-1],
        'execution_time': elapsed_time
    }

# 主程序执行
if __name__ == "__main__":
    # 设置随机种子以确保结果可重现
    np.random.seed(42)
    # 1. 运行标准梯度下降法
    print("\n" + "=" * 40)
    print("执行标准梯度下降法...")
    bgd_result = standard_gradient_descent(theta_init=100.0,
learning_rate=0.01, clip_value=1.0)
    print(f"最终θ值: {bgd_result['final_theta']:.8f}")
    print(f"目标函数值: {bgd_result['final_func']:.8f}")

```

```

print(f"迭代次数: {bgd_result['iterations']}")
print(f"是否收敛: {bgd_result['converged']}")
print(f"执行时间(秒): {bgd_result['execution_time']:.6f}")
print(f"迭代数据已保存到 bgd_result.xlsx 文件中")
# 2. 运行动量梯度下降法
print("\n" + "=" * 40)
print("执行动量梯度下降法...")
momentum_result = momentum_gradient_descent(theta_init=100.0,
learning_rate=0.01, gamma=0.9, clip_value=1.0)
print(f"最终 $\theta$ 值: {momentum_result['final_theta']:.8f}")
print(f"目标函数值: {momentum_result['final_func']:.8f}")
print(f"总迭代次数: {momentum_result['iterations']}")
if momentum_result['converged']:
    print(f"收敛于第 {momentum_result['iteration_converged']} 次迭
代")
else:
    print("未收敛")
print(f"执行时间(秒): {momentum_result['execution_time']:.6f}")
print(f"迭代数据已保存到 momentum_result.xlsx 文件中")
# 3. 运行带学习率衰减的 AMSGrad
print("\n" + "=" * 40)
print("运行带学习率衰减的优化 AMSGrad (高精度目标):")
amsgrad_result = amsgrad_optimized(
    theta_init=100.0,
    initial_learning_rate=0.05,
    beta1=0.9,
    beta2=0.999,
    epsilon=1e-8,
    clip_value=1.0,
    max_iter=10000,
    tol=1e-9,
    use_lr_decay=True,
    decay_rate=0.99,
    decay_steps=500
)
print(f"是否收敛: {amsgrad_result['converged']}")
print(f"迭代次数: {amsgrad_result['iterations']}")
if amsgrad_result['converged_at']:
    print(f"收敛于迭代: {amsgrad_result['converged_at']}")
print(f"最终  $\theta$ : {amsgrad_result['final_theta']:.10f}")
print(f"最终函数值: {amsgrad_result['final_func']:.10f}")
print(f"执行时间: {amsgrad_result['execution_time']:.4f} 秒")
print(f"迭代数据已保存到 amsgrad_result.xlsx 文件中")
# 4. 运行高精度 Adam 优化
print("\n" + "=" * 40)
print("运行高精度 Adam 优化 (从初始值 100 开始):")

```

```

adam_result = high_precision_adam(
    theta_init=100.0,
    learning_rate=0.05,
    beta1=0.9,
    beta2=0.999,
    epsilon=1e-10,
    max_iter=10000,
    tol=1e-3,
    use_lr_decay=True,
    decay_rate=0.998,
    decay_steps=200
)
print(f"是否收敛: {adam_result['converged']}")
print(f"迭代次数: {adam_result['iterations']}")
if adam_result['converged_at']:
    print(f"收敛于迭代: {adam_result['converged_at']}")
print(f"最终 Theta: {adam_result['final_theta']:.10f}")
print(f"最终函数值: {adam_result['final_func']:.10f}")
print(f"执行时间: {adam_result['execution_time']:.4f} 秒")
print(f"迭代数据已保存到 adam_result.xlsx 文件中")

```

附录 2

main.py

```

import sys
sys.path.append("./PaddleScience/")
sys.path.append('./3rd_lib')
sys.path.append("./model")
import argparse
import os
import csv
import pandas as pd
from timeit import default_timer
from typing import List
import numpy as np
import paddle
import yaml
from paddle.optimizer.lr import LRScheduler
from src.data import instantiate_datamodule
from src.networks import instantiate_network
from src.utils.average_meter import AverageMeter

```

```

from src.utils.dot_dict import DotDict
from src.utils.dot_dict import flatten_dict

class StepDecay(LRScheduler):
    def __init__(
        self, learning_rate, step_size, gamma=0.1, last_epoch=-1,
        verbose=False
    ):
        if not isinstance(step_size, int):
            raise TypeError(
                "The type of 'step_size' must be 'int', but received %s."
                % type(step_size)
            )
        if gamma >= 1.0:
            raise ValueError("gamma should be < 1.0.")

        self.step_size = step_size
        self.gamma = gamma
        super().__init__(learning_rate, last_epoch, verbose)

    def get_lr(self):
        i = self.last_epoch // self.step_size
        return self.base_lr * (self.gamma**i)

def instantiate_scheduler(config):
    if config.opt_scheduler == "CosineAnnealingLR":
        scheduler = paddle.optimizer.lr.CosineAnnealingDecay(
            config.lr, T_max=config.opt_scheduler_T_max
        )
    elif config.opt_scheduler == "StepLR":
        scheduler = StepDecay(
            config.lr,
            step_size=config.opt_step_size,
            gamma=config.opt_gamma
        )
    else:
        raise ValueError(f"Got {config.opt_scheduler=}")
    return scheduler

# loss function with rel/abs Lp loss
class LpLoss(object):
    def __init__(self, d=2, p=2, size_average=True, reduction=True):
        super(LpLoss, self).__init__()
        # Dimension and Lp-norm type are postive
        assert d > 0 and p > 0

        self.d = d
        self.p = p
        self.reduction = reduction

```

```

        self.size_average = size_average

    def abs(self, x, y):
        num_examples = x.size()[0]

        # Assume uniform mesh
        h = 1.0 / (x.size()[1] - 1.0)

        all_norms = (h ** (self.d / self.p)) * paddle.norm(
            x.reshape((num_examples, -1)) - y.reshape((num_examples,
-1))), self.p, 1
        )

        if self.reduction:
            if self.size_average:
                return paddle.mean(all_norms)
            else:
                return paddle.sum(all_norms)

        return all_norms

    def rel(self, x, y):
        diff_norms = paddle.norm(x-y, 2)
        y_norms = paddle.norm(y, self.p)

        if self.reduction:
            if self.size_average:
                return paddle.mean(diff_norms / y_norms)
            else:
                return paddle.sum(diff_norms / y_norms)

        return diff_norms / y_norms

    def __call__(self, x, y):
        return self.rel(x, y)

def set_seed(seed: int = 0):
    paddle.seed(seed)
    np.random.seed(seed)
    import random

    random.seed(seed)

def str2intlist(s: str) -> List[int]:
    return [int(item.strip()) for item in s.split(",")]

def parse_args(yaml="UnetShapeNetCar.yaml"):

```



```

parser = argparse.ArgumentParser()
parser.add_argument(
    "--config",
    type=str,
    default="configs/"+ yml,
    help="Path to the configuration file",
)
parser.add_argument(
    "--device",
    type=str,
    default="cuda",
    help="Device to use for training (cuda or cpu)",
)
parser.add_argument("--lr", type=float, default=None,
help="Learning rate")
parser.add_argument("--batch_size", type=int, default=None,
help="Batch size")
parser.add_argument("--num_epochs", type=int, default=None,
help="Number of epochs")
parser.add_argument(
    "--checkpoint",
    type=str,
    default=None,
    help="Path to the checkpoint file to resume training",
)
parser.add_argument(
    "--output",
    type=str,
    default="./output",
    help="Path to the output directory",
)
parser.add_argument(
    "--log",
    type=str,
    default="log",
    help="Path to the log directory",
)
parser.add_argument("--logger_types", type=str, nargs="+",
default=None)
parser.add_argument("--seed", type=int, default=0, help="Random
seed for training")
parser.add_argument("--model", type=str, default=None, help="Model
name")
parser.add_argument(
    "--sdf_spatial_resolution",
    type=str2intlist,
    default=None,
    help="SDF spatial resolution. Use comma to separate the values
e.g. 32,32,32.",
)

```

```

    args, _ = parser.parse_known_args()
    return args

def load_config(config_path):
    def include_constructor(loader, node):
        # Get the path of the current YAML file
        current_file_path = loader.name

        # Get the folder containing the current YAML file
        base_folder = os.path.dirname(current_file_path)

        # Get the included file path, relative to the current file
        included_file = os.path.join(base_folder,
loader.construct_scalar(node))

        # Read and parse the included file
        with open(included_file, "r") as file:
            return yaml.load(file, Loader=yaml.Loader)

    # Register the custom constructor for !include
    yaml.Loader.add_constructor("!include", include_constructor)

    with open(config_path, "r") as f:
        config = yaml.load(f, Loader=yaml.Loader)

    # Convert to dot dict
    config_flat = flatten_dict(config)
    config_flat = DotDict(config_flat)
    return config_flat

import re

def extract_numbers(s):
    return [int(digit) for digit in re.findall(r'\d+', s)]

def write_to_vtk(out_dict, point_data_pos="press on mesh points",
mesh_path=None, track=None):
    import meshio
    p = out_dict["pressure"]
    index = extract_numbers(mesh_path.name)[0]

    if track == "Dataset_1":
        index = str(index).zfill(3)
    elif track == "Track_B":
        index = str(index).zfill(4)

    print(f"Pressure shape for mesh {index} = {p.shape}")

```

```

if point_data_pos == "press on mesh points":
    mesh = meshio.read(mesh_path)
    mesh.point_data["p"] = p.numpy()
    if "pred wss_x" in out_dict:
        wss_x = out_dict["pred wss_x"]
        mesh.point_data["wss_x"] = wss_x.numpy()
elif point_data_pos == "press on mesh cells":
    points = np.load(mesh_path.parent / f"centroid_{index}.npy")
    npoint = points.shape[0]
    mesh = meshio.Mesh(
        points=points,
        cells=[("vertex",
np.arange(npoint).reshape(npoint, 1))]
    )
    mesh.point_data = {"p":p.numpy()}

    print(f"write : ./output/{mesh_path.parent.name}_{index}.vtk")
    mesh.write(f"./output/{mesh_path.parent.name}_{index}.vtk")

@paddle.no_grad()
def eval(model, datamodule, config, loss_fn=None, track="Dataset_1"):
    test_loader =
    datamodule.test_dataloader(batch_size=config.eval_batch_size,
shuffle=False, num_workers=0)
    data_list = []
    cd_list = []

    for i, data_dict in enumerate(test_loader):
        out_dict = model.eval_dict(data_dict, loss_fn=loss_fn,
decode_fn=datamodule.decode)
        if 'l2 eval loss' in out_dict:
            if i == 0:
                data_list.append(['id', 'l2 p'])
            else:
                data_list.append([i, float(out_dict['l2 eval loss'])])

        # TODO : you may write velocity into vtk, and analysis in your
report
        if config.write_to_vtk is True:
            print("datamodule.test_mesh_paths = ",
datamodule.test_mesh_paths[i])
            write_to_vtk(out_dict, config.point_data_pos,
datamodule.test_mesh_paths[i], track)

        # Your submit your npy to leaderboard here
        if "pressure" in out_dict:
            p = out_dict["pressure"].reshape((-1,)).astype(np.float32)
            test_indice = datamodule.test_indices[i]
            npy_leaderboard =

```

```

f"./output/{track}/press_{str(test_indice).zfill(3)}.npz"
    print(f"saving *.npz file for [{track}] leaderboard : ",
numpy_leaderboard)
    np.save(numpy_leaderboard, p)
    if "velocity" in out_dict:
        v
out_dict["velocity"].reshape((-1,3)).astype(np.float32)
        test_indice = datamodule.test_indices[i]
        numpy_leaderboard
f"./output/{track}/vel_{str(test_indice).zfill(3)}.npz"
    print(f"saving *.npz file for [{track}] leaderboard : ",
numpy_leaderboard)
    np.save(numpy_leaderboard, v)
    if "cd" in out_dict:
        v = out_dict["cd"].item()
        test_indice = datamodule.test_indices[i]
        cd_list.append([i, v])

    # check csv in ./output
    with open(f"./output/{config.project_name}.csv",
newline="") as file:
        writer = csv.writer(file)
        writer.writerows(data_list)

    if "cd" in out_dict:
        titles = ["", "Cd"]
        df = pd.DataFrame(cd_list, columns=titles)
        df.to_csv(f'./output/{track}/Answer.csv', index=False)
    return

def train(config):
    model = instantiate_network(config)
    datamodule = instantiate_datamodule(config)
    train_loader
    datamodule.train_dataloader(batch_size=config.batch_size,
shuffle=False)
    eval_dict = None
    # Initialize the optimizer
    scheduler = instantiate_scheduler(config)
    optimizer = paddle.optimizer.Adam(
        parameters=model.parameters(), learning_rate=scheduler,
weight_decay=1e-4
    )

    # Initialize the loss function
    loss_fn = LpLoss(size_average=True)
    L2 = []
    for ep in range(config.num_epochs):
        model.train()
        t1 = default_timer()

```

```

train_l2_meter = AverageMeter()
# train_reg = 0
for i, data_dict in enumerate(train_loader):
    optimizer.clear_grad()
    loss_dict = model.loss_dict(data_dict, loss_fn=loss_fn)
    loss = 0
    for k, v in loss_dict.items():
        loss = loss + v.mean()
    loss.backward()
    optimizer.step()
    train_l2_meter.update(loss.item())
scheduler.step()
t2 = default_timer()
print(
    f"Training epoch {ep} took {t2 - t1:.2f} seconds. L2 loss:
{train_l2_meter.avg:.4f}"
)

L2.append(train_l2_meter.avg)
if ep % config.eval_interval == 0 or ep == config.num_epochs -
1:
    # if you want to eval in Cars during training process, use
data in Training datasets
    # eval_dict = eval(model, datamodule, config, loss_fn)
    if eval_dict is not None:
        for k, v in eval_dict.items():
            print(f"Epoch: {ep} {k}: {v.item():.4f}")
    # Save the weights
    if ep % config.save_interval == 0 or ep == config.num_epochs -
1 and ep > 1:
        paddle.save(
            model.state_dict(),
            os.path.join("./output/",
f"model-{config.model}-{config.track}-{ep}.pdparams"),
        )

def load_yaml(file_name):
    args = parse_args(file_name)
    # args = parse_args("Unet_Velocity.yaml")
    config = load_config(args.config)

    # Update config with command line arguments
    for key, value in vars(args).items():
        if key != "config" and value is not None:
            config[key] = value

    # pretty print the config
    if paddle.distributed.get_rank() == 0:
        print(f"\n----- Config [{file_name}]
Table-----")

```

```

        for key, value in config.items():
            print("Key: {:<30} Val: {}".format(key, value))
        print("----- Config yaml Table-----\n")
    return config

def leader_board(config, track):
    os.makedirs(f"./output/{track}/", exist_ok=True)
    model = instantiate_network(config)
    checkpoint =
paddle.load(f"./output/model-{config.model}-{config.track}-{config.num_epochs - 1}.pdparams")
    model.load_dict(checkpoint)
    print(f"\n-----Starting Evaluation over [{config.track}]
-----")
    config.n_train = 1
    t1 = default_timer()

    config.mode="test"
    eval_dict = eval(
        model, instantiate_datamodule(config), config, loss_fn=lambda
x,y:0, track=track
    )
    t2 = default_timer()
    print(f"Inference over [Dataset_1 pressure] took {t2 - t1:.2f}
seconds.")

if __name__ == "__main__":
    os.makedirs("./output/", exist_ok=True)

    config_p = load_yaml("UnetShapeNetCar.yaml")
    train(config_p)
    leader_board(config_p, "Gen_Answer")

    config_cd = load_yaml("Unet_Cd.yaml")
    train(config_cd)
    leader_board(config_cd, "Gen_Answer")
    os.system(f"zip
-r ./output/Gen_Code.zip ./configs/ ./model/ ./README.md ./requirements.txt ./main.py ./main.ipynb")
    os.system(f"cd ./output && zip -r ./Gen_Answer.zip ./Gen_Answer")
    print("结果保存为/home/aistudio/MathorCup2025/output/Gen_Code.zip,
请上传平台完成提交")

```